LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – ABSTRACTION



OLEH

Nama : Lexi Mulia Yunaspi

NIM : 2511531006

Kelas : B

MATA KULIAH PEMOGRAMAN BERORIENTASI OBJEK

DOSEN PENGAMPU : Afdhal Dinilhak. . M.Kom,

ASISTEN PRAKTIKUM : EZZA ADDINI

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan Praktikum Pemrograman Berorientasi Objek Modul 6 dengan topik Abstraction ini dapat disusun. Laporan ini dibuat sebagai dokumentasi kegiatan praktikum dan pengerjaan tugas rumah yang membahas penerapan konsep abstraksi dalam bahasa pemrograman Java.

Materi dalam laporan ini mencakup abstract class, abstract method, interface, program to an interface, default method, static method, private method, pewarisan interface, final class, final method, serta static nested class, inner class, dan anonymous class. Laporan juga memuat pembahasan tugas rumah T1 sampai T4 sesuai petunjuk modul.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan agar laporan dan pemahaman penulis dapat menjadi lebih baik. Semoga laporan ini bermanfaat bagi pembaca.

Padang, 2026

Penulis

BAB I
PENDAHULUAN

LINK github :
https://github.com/lexi210919-dotcom/PrakPBO_B_2511531006

1.1 Latar Belakang

Pemrograman Berorientasi Objek (PBO) merupakan pendekatan pemrograman yang menyusun program berdasarkan objek dan class. Salah satu konsep penting dalam PBO adalah abstraction atau abstraksi. Abstraksi digunakan untuk menampilkan hal-hal penting yang perlu diketahui pengguna objek, sekaligus menyembunyikan detail proses yang tidak perlu ditampilkan.

Dalam bahasa Java, abstraksi dapat diterapkan melalui abstract class dan interface. Konsep ini membantu programmer membuat struktur program yang lebih teratur, mengurangi ketergantungan pada class tertentu, serta memudahkan pengembangan fitur. Modul Praktikum 6 menerapkan konsep tersebut melalui contoh proyek toko online UMKM yang mencakup produk, pengiriman, pembayaran, pesanan, dan notifikasi.

Melalui latihan L1 sampai L5 dan tugas rumah T1 sampai T4, mahasiswa mempraktikkan penggunaan berbagai bentuk abstraksi serta mengevaluasi desain program. Oleh karena itu, laporan ini disusun untuk menjelaskan proses dan hasil pengerjaan dengan bahasa yang sederhana.

1.2 Rumusan Masalah

1. Apa yang dimaksud dengan abstraction dalam pemrograman Java?
2. Bagaimana penggunaan abstract class dan interface pada proyek toko online?
3. Bagaimana penerapan default method, interface inheritance, final, dan nested class?
4. Bagaimana cara menilai dan memperbaiki desain program berdasarkan prinsip abstraksi?

1.3 Tujuan Praktikum

5. Memahami perbedaan abstract class dan interface serta kegunaannya.
6. Menerapkan abstraction pada program Java melalui latihan L1 sampai L5.
7. Mengembangkan interface menggunakan default, static, dan private method serta pewarisan interface.
8. Memahami penggunaan final class, final method, static nested class, inner class, dan anonymous class.
9. Mengevaluasi desain program dan mengusulkan perbaikan melalui tugas T1 sampai T4.

1.4 Manfaat Praktikum

Praktikum ini bermanfaat untuk meningkatkan pemahaman dalam merancang program Java yang terstruktur, mudah diuji, dan mudah dikembangkan. Selain itu, praktikum melatih kemampuan membaca pesan error compiler, melakukan perbaikan kode, serta memilih bentuk abstraksi yang sesuai dengan kebutuhan program.

BAB II
PEMBAHASAN

2.1 L1 – Abstract Class dan Abstract Method

Pada latihan pertama, materi yang diterapkan adalah abstract class dan abstract method. Abstract class digunakan sebagai class induk yang menyediakan atribut serta perilaku umum untuk class turunannya. Dalam program toko sederhana, class Product menjadi induk bagi produk fisik dan produk digital. Kedua jenis produk memiliki SKU, nama, dan harga, tetapi memiliki kategori serta informasi khusus yang berbeda.

1. Product.java

```
2. package shop.model;
3. 
4. public abstract class Product {
5.     private final String sku;
6.     private final String name;
7.     private final long price;
8. 
9.     protected Product(String sku, String name, long price) {
10.         if (sku == null || sku.isBlank()) {
11.             throw new IllegalArgumentException("SKU must not be blank");
12.         }
13.         if (name == null || name.isBlank()) {
14.             throw new IllegalArgumentException("Name must not be
   blank");
15.         }
16.         if (price < 0) {
17.             throw new IllegalArgumentException("Price must not be
   negative");
18.         }
19. 
20.         this.sku = sku;
21.         this.name = name;
22.         this.price = price;
23.     }
24. 
25.     public String getSku() {
26.         return sku;
27.     }
28. 
29.     public String getName() {
30.         return name;
31.     }
32. 
33.     public long getPrice() {
34.         return price;
35.     }
```

```
36.
37.     public long getFinalPrice() {
38.         return price;
39.     }
40.
41.     public abstract String category();
42.
43.     public String getInfo() {
44.         return "[" + category() + "] " + name + " - Rp" +
    getFinalPrice();
45.     }
46. }
```

Class Product merupakan abstract class yang menjadi dasar bagi seluruh jenis produk. Class ini memiliki atribut sku, name, dan price yang dibuat private final agar nilainya tidak dapat diubah secara langsung setelah objek dibuat. Constructor digunakan untuk mengisi atribut sekaligus melakukan validasi data. Method category() dan getInfo() merupakan abstract method sehingga setiap class turunan wajib memberikan implementasinya sendiri. Dengan demikian, class Product menyediakan struktur umum tanpa menentukan seluruh detail dari setiap jenis produk.

2. PhysicalProduct.java

```
3.  package shop.model;
4.
5.  import shop.shipping.Shippable;
6.
7.  public class PhysicalProduct extends Product implements Shippable {
8.
9.      private final int weightGrams;
10.
11.     public PhysicalProduct(String sku, String name, long price, int
    weightGrams) {
12.         super(sku, name, price);
13.
14.         if (weightGrams <= 0) {
15.             throw new IllegalArgumentException("Weight must be
    positive");
16.         }
17.
18.         this.weightGrams = weightGrams;
19.     }
20.
21.     @Override
22.     public int getWeightGrams() {
23.         return weightGrams;
24.     }
25.
26.     @Override
```

```
27.     public String category() {
28.         return "Fisik";
29.     }
30. }
```

Class PhysicalProduct merupakan turunan dari class Product yang digunakan untuk merepresentasikan produk berbentuk fisik. Class ini memiliki atribut tambahan berupa weightGrams untuk menyimpan berat barang dalam satuan gram. Constructor menggunakan super() untuk memanggil constructor dari class induk, kemudian memvalidasi berat produk. Class ini juga mengimplementasikan interface Shippable karena produk fisik membutuhkan proses pengiriman. Method category() menunjukkan bahwa produk termasuk kategori fisik, sedangkan getInfo() menampilkan informasi produk beserta beratnya.

3. DigitalProduct.java

```
4.  package shop.model;
5.
6.  import shop.discount.Discountable;
7.
8.  public class DigitalProduct extends Product implements Discountable {
9.
10.     private static final int DIGITAL_DISCOUNT_PERCENT = 10;
11.     private final String downloadUrl;
12.
13.     public DigitalProduct(String sku, String name, long price, String
    downloadUrl) {
14.         super(sku, name, price);
15.
16.         if (downloadUrl == null || downloadUrl.isBlank()) {
17.             throw new IllegalArgumentException("Download URL must not be
    blank");
18.         }
19.
20.         this.downloadUrl = downloadUrl;
21.     }
22.
23.     public String getDownloadUrl() {
24.         return downloadUrl;
25.     }
26.
27.     @Override
28.     public int discountPercent() {
29.         return DIGITAL_DISCOUNT_PERCENT;
30.     }
31.
32.     @Override
33.     public long getFinalPrice() {
34.         return applyDiscount(getPrice());
```

```
    }

    @Override
    public String category() {
        return "Digital";
    }
}
```

Class DigitalProduct merupakan turunan dari Product yang digunakan untuk produk digital seperti e-book atau file unduhan. Berbeda dengan produk fisik, produk digital memiliki atribut downloadUrl untuk menyimpan alamat unduhan dan tidak membutuhkan berat pengiriman. Constructor melakukan validasi terhadap URL agar tidak kosong. Method category() mengembalikan kategori Digital, sedangkan getInfo() menampilkan nama produk, harga, dan alamat unduhannya. Pada latihan L3, class ini akan dikembangkan kembali dengan menambahkan kemampuan diskon melalui interface Discountable.

4. **ShippingService.java**

```
package shop.shipping;

public abstract class ShippingService {

    private final String name;

    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be
blank");
        }

        this.name = name;
    }

    public String getName() {
        return name;
    }

    protected abstract long ratePerKg();

    public final long calculateCost(int weightGrams) {
        if (weightGrams <= 0) {
            return 0;
        }

        long kilograms = (weightGrams + 999L) / 1000;
        return kilograms * ratePerKg();
    }
}
```

Class ShippingService merupakan abstract class yang menjadi dasar bagi berbagai jenis layanan pengiriman. Atribut name digunakan untuk menyimpan nama layanan pengiriman. Method ratePerKg() dibuat abstrak karena setiap layanan memiliki tarif yang berbeda. Sementara itu, method calculateCost() digunakan untuk menghitung ongkos kirim berdasarkan berat barang. Perhitungan dilakukan dengan membulatkan berat ke atas menjadi kilogram, kemudian mengalikannya dengan tarif per kilogram. Pada latihan L4, method calculateCost() akan ditambahkan modifier final.

5. RegularShipping.java

```
6. package shop.shipping;
7. 
8. public class RegularShipping extends ShippingService {
9. 
10.     public RegularShipping() {
11.         super("Reguler");
12.     }
13. 
14.     @Override
15.     protected long ratePerKg() {
16.         return 10_000;
17.     }
18.}
```

Class RegularShipping merupakan turunan dari ShippingService yang digunakan untuk layanan pengiriman reguler. Constructor memanggil constructor induk menggunakan super() dengan nama layanan Reguler. Method ratePerKg() diimplementasikan untuk menentukan tarif pengiriman sebesar Rp10.000 per kilogram. Dengan adanya subclass ini, tarif pengiriman reguler dapat ditentukan tanpa mengubah rumus perhitungan yang berada pada class induk.

6. ExpressShipping.java

```
7. package shop.shipping;
8. 
9. public class ExpressShipping extends ShippingService {
10. 
11.     public ExpressShipping() {
12.         super("Ekspres");
13.     }
14. 
15.     @Override
16.     protected long ratePerKg() {
17.         return 18_000;
18.     }
19.}
```

Class ExpressShipping merupakan turunan dari ShippingService yang digunakan untuk layanan pengiriman ekspres. Constructor menetapkan nama layanan Ekspres melalui constructor induk. Method ratePerKg() mengembalikan tarif sebesar Rp18.000 per kilogram. Perbedaan tarif antara pengiriman reguler dan ekspres menunjukkan bahwa setiap subclass dapat memberikan implementasi berbeda terhadap abstract method yang diwarisinya.

2.2 L2 – Interface dan Program to an Interface

Pada latihan kedua, program dikembangkan dengan menggunakan interface. Interface berfungsi sebagai kontrak yang menentukan method yang harus dimiliki oleh class yang mengimplementasikannya. Penggunaan interface membuat program lebih fleksibel karena class layanan dapat menggunakan objek berdasarkan kemampuan yang dimiliki, bukan hanya berdasarkan jenis class tertentu.

1. Shippable.java

```
2. package shop.shipping;
3.
4. public interface Shippable {
5.     int getWeightGrams();
6. }
```

Interface Shippable digunakan untuk menentukan kemampuan suatu produk agar dapat dikirim. Interface ini memiliki method getWeightGrams() yang harus diimplementasikan oleh class yang menggunakannya. Dalam program ini, PhysicalProduct mengimplementasikan Shippable karena barang fisik memiliki berat dan membutuhkan pengiriman. Sementara itu, produk digital tidak perlu mengimplementasikan interface tersebut.

2. PaymentMethod.java

```
3. package shop.payment;
4.
5. public interface PaymentMethod {
6.     String getName();
7.
8.     boolean pay(long amount);
9. }
```

Interface PaymentMethod digunakan sebagai kontrak untuk berbagai metode pembayaran.Method getName() berfungsi untuk mendapatkan nama metode pembayaran,sedangkan method pay() digunakan untuk menjalankan proses

pembayaran berdasarkan jumlah uang yang harus dibayarkan.Setiap class pembayaran harus mengimplementasikan kedua method tersebut agar dapat digunakan dalam proses checkout.

3. EWalletPayment.java

```
4.  package shop.payment;
5.
6.  public class EWalletPayment implements RefundablePayment {
7.
8.      private final String name;
9.      private long balance;
10.
11.     public EWalletPayment(String name, long balance) {
12.         if (name == null || name.isBlank()) {
13.             throw new IllegalArgumentException("Name must not be
    blank");
14.         }
15.         if (balance < 0) {
16.             throw new IllegalArgumentException("Balance must not be
    negative");
17.         }
18.
19.         this.name = name;
20.         this.balance = balance;
21.     }
22.
23.     @Override
24.     public String getName() {
25.         return name;
26.     }
27.
28.     public long getBalance() {
29.         return balance;
30.     }
31.
32.     @Override
33.     public boolean pay(long amount) {
34.         if (amount <= 0 || amount > balance) {
35.             return false;
36.         }
37.
38.         balance -= amount;
39.         return true;
40.     }
41.
42.     @Override
43.     public boolean refund(long amount) {
44.         if (amount <= 0) {
45.             return false;
46.         }
47.
48.         balance += amount;
49.         return true;
```

```
    }
}
```

Class EWalletPayment mengimplementasikan interface PaymentMethod untuk menyediakan pembayaran menggunakan saldo dompet digital. Class ini memiliki atribut name untuk menyimpan nama e-wallet dan balance untuk menyimpan saldo yang tersedia. Method pay() memeriksa apakah jumlah pembayaran valid dan tidak melebihi saldo. Apabila pembayaran berhasil, saldo akan dikurangi sesuai nominal transaksi. Jika saldo tidak mencukupi, pembayaran akan ditolak.

4. BankTransferPayment.java

```java
package shop.payment;

public class BankTransferPayment implements PaymentMethod {

    private final String bankName;

    public BankTransferPayment(String bankName) {
        if (bankName == null || bankName.isBlank()) {
            throw new IllegalArgumentException("Bank name must not be blank");
        }

        this.bankName = bankName;
    }

    @Override
    public String getName() {
        return "Transfer Bank (" + bankName + ")";
    }

    @Override
    public boolean pay(long amount) {
        return amount > 0;
    }
}
```

Class BankTransferPayment merupakan implementasi lain dari interface PaymentMethod. Class ini digunakan untuk menyimulasikan pembayaran melalui transfer bank dengan menyimpan nama bank pada atribut bankName.Method getName() mengembalikan nama metode pembayaran berupa transfer bank. Method pay() memeriksa apakah nominal pembayaran lebih dari nol. Dengan menerapkan interface yang sama, pembayaran melalui e-wallet dan transfer bank dapat digunakan oleh layanan checkout dengan cara yang seragam.

5. CheckoutService.java

```
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.payment.RefundablePayment;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;

public class CheckoutService {

    public long subtotal(List<Product> items) {
        long total = 0;

        for (Product product : items) {
            total += product.getFinalPrice();
        }

        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;

        for (Product product : items) {
            if (product instanceof Shippable shippable) {
                grams += shippable.getWeightGrams();
            }
        }

        return grams;
    }

    public long total(List<Product> items, ShippingService shipping) {
        return subtotal(items) +
shipping.calculateCost(totalWeight(items));
    }

    public boolean checkout(
            List<Product> items,
            ShippingService shipping,
            PaymentMethod payment) {

        if (items == null || items.isEmpty()) {
            return false;
        }

        return payment.pay(total(items, shipping));
    }

    public boolean refund(RefundablePayment payment, long amount) {
        return payment.refund(amount);
    }
}
```

Class CheckoutService bertanggung jawab mengelola proses checkout. Method subtotal() menghitung jumlah harga akhir seluruh produk, sedangkan totalWeight() menghitung berat produk yang mengimplementasikan Shippable.Method total() menjumlahkan subtotal dengan ongkos pengiriman.Method checkout() menerima objek bertipe PaymentMethod dan ShippingService,sehingga proses pembayaran tidak bergantung pada satu jenis metode pembayaran atau layanan pengiriman saja. Penerapan ini merupakan contoh konsep program to an interface.

2.3 L3 – Default, Static, Private Method, dan Interface Inheritance

Pada latihan ketiga, program dikembangkan dengan menambahkan beberapa jenis method pada interface, yaitu default method, static method, dan private method. Selain itu, dipraktikkan juga pewarisan interface menggunakan kata kunci extends. Penerapannya digunakan untuk menambahkan fitur diskon pada produk digital, pengembalian dana, serta pembayaran menggunakan cicilan.

1. **Discountable.java**

```
2.  package shop.discount;
3.
4.  public interface Discountable {
5.
6.      int discountPercent();
7.
8.      default long applyDiscount(long price) {
9.          return price - percentOf(price, safePercent());
10.     }
11.
12.     static long percentOf(long amount, int percent) {
13.         return amount * percent / 100;
14.     }
15.
16.     private int safePercent() {
17.         int percent = discountPercent();
18.
19.         if (percent < 0) {
20.             return 0;
21.         }
22.
23.         return Math.min(percent, 100);
24.     }
25. }
```

Interface Discountable digunakan untuk memberikan kemampuan diskon kepada produk.Method discountPercent() menentukan persentase diskon yang

diberikan oleh class implementasi.Default method applyDiscount() menghitung harga setelah dikurangi diskon.Static method percentOf() digunakan untuk menghitung nominal potongan harga,sedangkan private method safePercent() membatasi persentase diskon agar tetap berada pada rentang 0 sampai 100 persen.Dengan demikian, perhitungan diskon dapat dilakukan secara lebih terstruktur dan aman.

2. DigitalProduct.java

```
3. package shop.model;
4. 
5. import shop.discount.Discountable;
6. 
7. public class DigitalProduct extends Product implements Discountable {
8. 
9.     private static final int DIGITAL_DISCOUNT_PERCENT = 10;
10.     private final String downloadUrl;
11. 
12.     public DigitalProduct(String sku, String name, long price, String
    downloadUrl) {
13.         super(sku, name, price);
14. 
15.         if (downloadUrl == null || downloadUrl.isBlank()) {
16.             throw new IllegalArgumentException("Download URL must not be
    blank");
17.         }
18. 
19.         this.downloadUrl = downloadUrl;
20.     }
21. 
22.     public String getDownloadUrl() {
23.         return downloadUrl;
24.     }
25. 
26.     @Override
27.     public int discountPercent() {
28.         return DIGITAL_DISCOUNT_PERCENT;
29.     }
30. 
31.     @Override
32.     public long getFinalPrice() {
33.         return applyDiscount(getPrice());
34.     }
35. 
36.     @Override
37.     public String category() {
38.         return "Digital";
39.     }
40. }
```

Pada L3,class DigitalProduct dikembangkan dengan mengimplementasikan interface Discountable. Konstanta DIGITAL_DISCOUNT_PERCENT digunakan

untuk menentukan diskon produk digital sebesar 10 persen. Method discountPercent() mengembalikan nilai diskon tersebut, sedangkan method getFinalPrice() menggunakan applyDiscount() untuk menghitung harga setelah diskon.Sebagai contoh, apabila harga awal produk digital sebesar Rp50.000, maka diskon 10 persen menghasilkan potongan sebesar Rp5.000 sehingga harga akhirnya menjadi Rp45.000.

3. RefundablePayment.java

```
4. package shop.payment;
5. 
6. public interface RefundablePayment extends PaymentMethod {
7. 
8.     boolean refund(long amount);
9. }
```

Interface RefundablePayment merupakan interface turunan dari PaymentMethod. Pewarisan dilakukan menggunakan kata kunci extends, sehingga interface ini tetap memiliki kontrak getName() dan pay() dari interface induknya. Selain itu, RefundablePayment menambahkan method refund() untuk mengatur proses pengembalian dana. Dengan cara ini, tidak semua metode pembayaran harus mendukung refund, tetapi hanya metode pembayaran yang mengimplementasikan RefundablePayment.

4. InstallmentPayment.java

```
5. package shop.payment;
6. 
7. public interface InstallmentPayment extends PaymentMethod {
8. 
9.     int MAX_MONTHS = 12;
10. 
11.     default long monthlyInstallment(long amount, int months) {
12.         if (months <= 0 || months > MAX_MONTHS) {
13.             throw new IllegalArgumentException("Invalid installment
    months");
14.         }
15. 
16.         return (amount + months - 1) / months;
17.     }
18.}
```

Interface InstallmentPayment digunakan untuk memberikan kemampuan pembayaran secara cicilan. Interface ini mewarisi PaymentMethod sehingga class yang mengimplementasikannya juga harus menyediakan method pembayaran. Method maxInstallmentMonths() menentukan batas maksimal jumlah bulan

cicilan. Default method monthlyInstallment() memvalidasi nominal pembayaran dan jumlah bulan, kemudian menghitung jumlah cicilan setiap bulan dengan pembulatan ke atas. Contohnya, pembayaran sebesar Rp268.000 selama tiga bulan menghasilkan cicilan sebesar Rp89.334 per bulan.

5. EWalletPayment.java

```
6.  package shop.payment;
7.
8.  public class EWalletPayment implements RefundablePayment {
9.
10.     private final String name;
11.     private long balance;
12.
13.     public EWalletPayment(String name, long balance) {
14.         if (name == null || name.isBlank()) {
15.             throw new IllegalArgumentException("Name must not be
    blank");
16.         }
17.         if (balance < 0) {
18.             throw new IllegalArgumentException("Balance must not be
    negative");
19.         }
20.
21.         this.name = name;
22.         this.balance = balance;
23.     }
24.
25.     @Override
26.     public String getName() {
27.         return name;
28.     }
29.
30.     public long getBalance() {
31.         return balance;
32.     }
33.
34.     @Override
35.     public boolean pay(long amount) {
36.         if (amount <= 0 || amount > balance) {
37.             return false;
38.         }
39.
40.         balance -= amount;
41.         return true;
42.     }
43.
44.     @Override
45.     public boolean refund(long amount) {
46.         if (amount <= 0) {
47.             return false;
48.         }
49.
```

```
50.         balance += amount;
51.         return true;
52.     }
53. }
```

Pada L3, class EWalletPayment mengalami perubahan dengan mengimplementasikan RefundablePayment sebagai pengganti langsung dari PaymentMethod. Karena RefundablePayment merupakan turunan PaymentMethod, class ini tetap harus memiliki method getName() dan pay(). Selain itu, ditambahkan method refund() untuk mengembalikan dana ke saldo e-wallet. Apabila nominal refund lebih dari nol, saldo akan bertambah sesuai jumlah pengembalian dana. Jika nominal tidak valid, proses refund akan ditolak.

6. CreditCardPayment.java

```
7.  package shop.payment;
8.
9.  public class CreditCardPayment implements RefundablePayment,
    InstallmentPayment {
10.
11.     private final String cardHolder;
12.     private long limit;
13.     private long used;
14.
15.     public CreditCardPayment(String cardHolder, long limit) {
16.         if (cardHolder == null || cardHolder.isBlank()) {
17.             throw new IllegalArgumentException("Card holder must not be
    blank");
18.         }
19.         if (limit < 0) {
20.             throw new IllegalArgumentException("Limit must not be
    negative");
21.         }
22.
23.         this.cardHolder = cardHolder;
24.         this.limit = limit;
25.     }
26.
27.     @Override
28.     public String getName() {
29.         return "Kartu Kredit (" + cardHolder + ")";
30.     }
31.
32.     @Override
33.     public boolean pay(long amount) {
34.         if (amount <= 0 || amount > limit - used) {
35.             return false;
36.         }
37.
38.         used += amount;
39.         return true;
40.     }
```

```
41.
42.     @Override
43.     public boolean refund(long amount) {
44.         if (amount <= 0 || amount > used) {
45.             return false;
46.         }
47.
48.         used -= amount;
49.         return true;
50.     }
51.}
```

Class CreditCardPayment mengimplementasikan dua interface sekaligus, yaitu RefundablePayment dan InstallmentPayment. Class ini memiliki atribut cardHolder untuk menyimpan nama pemegang kartu, creditLimit untuk menyimpan batas kredit, dan usedCredit untuk mencatat jumlah kredit yang telah digunakan. Method pay() memeriksa apakah pembayaran tidak melebihi limit kartu. Method refund() mengurangi jumlah kredit yang telah digunakan apabila pengembalian dana berhasil. Method maxInstallmentMonths() menentukan batas maksimal cicilan selama 12 bulan. Dengan demikian, satu class dapat memiliki kemampuan pembayaran, refund, dan cicilan sekaligus.

7. Tambahan method refund() pada CheckoutService.java

```
8.   public boolean refund(RefundablePayment payment, long amount) {
9.         return payment.refund(amount);
10.     }
11.}
12.
```

Tambahan ini masuk method ke dalam class CheckoutService yang dibuat pada L2. Letakkan method ini di dalam kurung kurawal class CheckoutService, sebelum kurung kurawal penutup terakhir.

Method refund() digunakan untuk meneruskan proses pengembalian dana kepada objek yang mengimplementasikan RefundablePayment. Parameter menggunakan tipe interface, bukan class pembayaran tertentu. Hal ini membuat method dapat digunakan oleh e-wallet maupun kartu kredit selama keduanya mendukung refund. Sebaliknya, BankTransferPayment tidak dapat diberikan sebagai parameter karena class tersebut hanya mengimplementasikan PaymentMethod, bukan RefundablePayment.

2.4 L4 – Final Method dan Final Class

Pada latihan keempat, materi yang dipelajari adalah penggunaan modifier final pada method dan class. Modifier final digunakan untuk membatasi perubahan atau pewarisan pada bagian tertentu dari program. Dalam latihan ini, modifier tersebut diterapkan pada perhitungan ongkos kirim dan class alamat agar aturan perhitungan serta data alamat tetap terjaga.

1. **Perubahan calculateCost() pada ShippingService.java**

```
2.   public final long calculateCost(int weightGrams) {
3.         if (weightGrams <= 0) {
4.             return 0;
5.         }
6.
7.         long kilograms = (weightGrams + 999L) / 1000;
8.         return kilograms * ratePerKg();
9.     }
10.}
```

Pada file ShippingService.java, method calculateCost() diubah menjadi final. Bagian method yang digunakan pada L4 adalah sebagai berikut.Penambahan modifier final menyebabkan method calculateCost() tidak dapat di-override oleh class turunannya. Dengan demikian, class RegularShipping dan ExpressShipping tetap dapat menentukan tarif melalui method ratePerKg(), tetapi tidak dapat mengubah rumus perhitungan ongkos kirim. Penggunaan final method membantu menjaga konsistensi aturan perhitungan pada seluruh jenis layanan pengiriman.

2. **Address.java**

```
3.  package shop.model;
4.
5.  public final class Address {
6.
7.      private final String street;
8.      private final String city;
9.      private final String postalCode;
10.
11.     public Address(String street, String city, String postalCode) {
12.         if (street == null || street.isBlank()
13.                 || city == null || city.isBlank()
14.                 || postalCode == null || postalCode.isBlank()) {
15.             throw new IllegalArgumentException("Address fields must not
    be blank");
16.         }
17.
18.         this.street = street;
```

```
        this.city = city;
        this.postalCode = postalCode;
    }

    public String getStreet() {
        return street;
    }

    public String getCity() {
        return city;
    }

    public String getPostalCode() {
        return postalCode;
    }

    @Override
    public String toString() {
        return street + ", " + city + " " + postalCode;
    }
}
```

Class Address dibuat menggunakan modifier final sehingga tidak dapat diwarisi oleh class lain. Atribut street, city, dan postalCode dibuat private final agar nilainya tidak dapat diubah setelah proses pembuatan objek. Constructor melakukan validasi untuk memastikan setiap bagian alamat telah diisi. Class ini hanya menyediakan getter tanpa setter sehingga data alamat bersifat tetap atau immutable. Method toString() digunakan untuk menggabungkan jalan, kota, dan kode pos menjadi satu teks alamat.

2.5 L5 – Static Nested Class, Inner Class, dan Anonymous Class

Pada latihan kelima, materi yang dipelajari adalah static nested class,inner class,dan anonymous class.Ketiga jenis class tersebut digunakan untuk mengelompokkan bagian program yang memiliki hubungan dengan class utama. Dalam program toko, static nested class digunakan untuk menyimpan item pesanan, inner class digunakan untuk membuat struk, sedangkan anonymous class digunakan untuk membuat metode pembayaran voucher tanpa membuat file class baru.

1. **Order.java**

```
package shop.model;

import java.util.ArrayList;
```

```
5. import java.util.Collections;
6. import java.util.List;
7.
8. public class Order {
9.
10.     private final String orderId;
11.     private final Address shippingAddress;
12.     private final List<Line> lines = new ArrayList<>();
13.
14.     public Order(String orderId, Address shippingAddress) {
15.         if (orderId == null || orderId.isBlank() || shippingAddress ==
    null) {
16.             throw new IllegalArgumentException("Order id and address are
    required");
17.         }
18.
19.         this.orderId = orderId;
20.         this.shippingAddress = shippingAddress;
21.     }
22.
23.     public String getOrderId() {
24.         return orderId;
25.     }
26.
27.     public Address getShippingAddress() {
28.         return shippingAddress;
29.     }
30.
31.     public void addLine(Product product, int quantity) {
32.         lines.add(new Line(product, quantity));
33.     }
34.
35.     public List<Line> getLines() {
36.         return Collections.unmodifiableList(lines);
37.     }
38.
39.     public long getItemsTotal() {
40.         long total = 0;
41.
42.         for (Line line : lines) {
43.             total += line.subtotal();
44.         }
45.
46.         return total;
47.     }
48.
49.     public Receipt createReceipt() {
50.         return new Receipt();
51.     }
52.
53.     public static class Line {
54.         private final Product product;
55.         private final int quantity;
56.
57.         public Line(Product product, int quantity) {
58.             if (product == null || quantity <= 0) {
```

```
59.                 throw new IllegalArgumentException("Product and positive
    quantity are required");
60.             }
61.
62.             this.product = product;
63.             this.quantity = quantity;
64.         }
65.
66.         public Product getProduct() {
67.             return product;
68.         }
69.
70.         public int getQuantity() {
71.             return quantity;
72.         }
73.
74.         public long subtotal() {
75.             return product.getFinalPrice() * quantity;
76.         }
77.     }
78.
79.     public class Receipt {
80.
81.         public String toText() {
82.             StringBuilder text = new StringBuilder();
83.
84.             text.append("STRUK PESANAN
    ").append(Order.this.orderId).append("\n");
85.
86.             for (Line line : lines) {
87.                 text.append("- ")
88.                     .append(line.getProduct().getName())
89.                     .append(" x")
90.                     .append(line.getQuantity())
91.                     .append(" = Rp")
92.                     .append(line.subtotal())
93.                     .append("\n");
94.             }
95.
96.             text.append("Total barang:
    Rp").append(getItemsTotal()).append("\n");
97.             text.append("Kirim ke:
    ").append(Order.this.shippingAddress);
98.
99.             return text.toString();
100.            }
101.        }
102.    }
```

Class Order digunakan untuk menyimpan informasi pesanan yang terdiri dari ID pesanan, alamat pengiriman, dan daftar produk yang dibeli. Di dalam class ini terdapat static nested class Line yang digunakan untuk merepresentasikan satu baris item pesanan. Class Line memiliki atribut produk dan jumlah pembelian serta method subtotal() untuk menghitung harga produk berdasarkan jumlahnya. Selain

itu, terdapat inner class Receipt yang digunakan untuk menyusun teks struk pesanan. Inner class tersebut dapat mengakses atribut milik objek Order melalui Order.this, sehingga dapat menampilkan ID pesanan dan alamat pengiriman. Method getItemsTotal() menghitung total seluruh item, sedangkan createReceipt() membuat objek struk.

2. ShopApp.java

```
package shop.app;

import java.util.List;

import shop.model.Address;
import shop.model.DigitalProduct;
import shop.model.Order;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.service.CheckoutService;
import shop.shipping.RegularShipping;

public class ShopApp {

    public static void main(String[] args) {

        Product coffee = new PhysicalProduct(
                "P001", "Kopi Arabika Gayo 250g", 85_000, 250);

        Product ebook = new DigitalProduct(
                "D001", "E-book Resep Kue Nusantara", 50_000,
                "https://example.com/ebook");

        CheckoutService checkout = new CheckoutService();
        RegularShipping regular = new RegularShipping();

        Address address = new Address(
                "Jl. Moh Hatta No. 10", "Padang", "25121");

        Order order = new Order("ORD-0001", address);
        order.addLine(coffee, 2);
        order.addLine(ebook, 1);

        Order.Receipt receipt = order.new Receipt();
        System.out.println(receipt.toText());

        PaymentMethod voucher = new PaymentMethod() {
            private long remaining = 50_000;
```

```
42.
43.             @Override
44.             public String getName() {
45.                 return "Voucher Promo Hari Ini";
46.             }
47.
48.             @Override
49.             public boolean pay(long amount) {
50.                 if (amount <= 0 || amount > remaining) {
51.                     return false;
52.                 }
53.
54.                 remaining -= amount;
55.                 return true;
56.             }
57.         };
58.
59.         System.out.println("\n" + voucher.getName()
60.                 + " untuk e-book berhasil? "
61.                 + checkout.checkout(List.of(ebook), regular, voucher));
62.     }
63. }
```

Class ShopApp merupakan class utama yang digunakan untuk menjalankan program toko. Pada bagian awal, program membuat objek produk fisik dan digital, kemudian menampilkan informasi produk menggunakan method getInfo(). Selanjutnya, program membuat objek CheckoutService serta layanan pengiriman reguler dan ekspres untuk menghitung subtotal, berat barang, dan total pembayaran. Program juga menguji beberapa metode pembayaran, yaitu e-wallet, transfer bank, dan kartu kredit. Setelah itu, dilakukan pengujian refund, pembuatan alamat, penambahan produk ke dalam pesanan, serta pencetakan struk menggunakan inner class Receipt. Pada bagian akhir, dibuat anonymous class yang mengimplementasikan PaymentMethod untuk menyimulasikan voucher promo dengan saldo Rp50.000.

3. Tambahan pada CheckoutService.java untuk L3

```
4.  package shop.service;
5.
6.  import java.util.List;
7.  import shop.model.Product;
8.  import shop.payment.PaymentMethod;
9.  import shop.payment.RefundablePayment;
10. import shop.shipping.Shippable;
11. import shop.shipping.ShippingService;
```

```
12.
13. public class CheckoutService {
14.
15.     public long subtotal(List<Product> items) {
16.         long total = 0;
17.
18.         for (Product product : items) {
19.             total += product.getFinalPrice();
20.         }
21.
22.         return total;
23.     }
24.
25.     public int totalWeight(List<Product> items) {
26.         int grams = 0;
27.
28.         for (Product product : items) {
29.             if (product instanceof Shippable shippable) {
30.                 grams += shippable.getWeightGrams();
31.             }
32.         }
33.
34.         return grams;
35.     }
36.
37.     public long total(List<Product> items, ShippingService shipping) {
38.         return subtotal(items) +
    shipping.calculateCost(totalWeight(items));
39.     }
40.
41.     public boolean checkout(
42.             List<Product> items,
43.             ShippingService shipping,
44.             PaymentMethod payment) {
45.
46.         if (items == null || items.isEmpty()) {
47.             return false;
48.         }
49.
50.         return payment.pay(total(items, shipping));
51.     }
52.
53.     public boolean refund(RefundablePayment payment, long amount) {
54.         return payment.refund(amount);
55.     }
56. }
```

Agar pemanggilan refund di dalam ShopApp dapat dijalankan, file CheckoutService.java perlu memiliki method refund. Berikut versi lengkap

CheckoutService.java setelah ditambahkan fitur refund.Pada versi ini, CheckoutService telah memiliki method refund() yang menerima objek bertipe RefundablePayment. Method tersebut meneruskan proses pengembalian dana kepada metode pembayaran yang digunakan. Dengan demikian, e-wallet dan kartu kredit dapat melakukan refund karena keduanya mengimplementasikan interface tersebut. Sementara itu, transfer bank tidak dapat digunakan untuk refund karena tidak memenuhi kontrak RefundablePayment.

2.6 T1 – Evolusi API dan Konflik Diamond

Tugas ini membahas pengembangan interface pada Java, penggunaan default method, biaya administrasi pembayaran, serta penyelesaian konflik ketika sebuah class mengimplementasikan dua interface yang memiliki method default dengan nama sama.

1. **PaymentMethod.java`**

```
2. package shop.payment;
3. 
4. public interface PaymentMethod {
5.     String getName();
6. 
7.     boolean pay(long amount);
8. 
9.     default long adminFee(long amount) {
10.         return 0;
11.     }
12.}
```

Interface `PaymentMethod` digunakan sebagai aturan dasar untuk semua metode pembayaran. Setiap class yang menggunakan interface ini harus memiliki method `getName()` untuk mendapatkan nama pembayaran dan `pay()` untuk menjalankan proses pembayaran. Selain itu, terdapat method `adminFee()` yang digunakan untuk menghitung biaya administrasi. Method ini menggunakan kata kunci `default` sehingga sudah memiliki nilai awal sebesar 0. Dengan begitu, metode pembayaran yang tidak memiliki biaya administrasi tidak perlu membuat ulang method tersebut.

2. RefundablePayment.java

```
3. package shop.payment;
4.
5. public interface RefundablePayment extends PaymentMethod {
6.
7.     boolean refund(long amount);
8.
9.     default String policy() {
10.        return "Refund maksimal 7 hari";
11.    }
12.}
```

Interface `RefundablePayment` merupakan pengembangan dari interface `PaymentMethod` yang menambahkan kemampuan pengembalian dana atau refund. Method `refund()` digunakan untuk menjalankan proses pengembalian uang sesuai jumlah yang dimasukkan. Interface ini juga memiliki method `policy()` dengan nilai awal berupa informasi bahwa refund maksimal dilakukan dalam waktu 7 hari. Dengan adanya interface ini, hanya metode pembayaran yang mendukung pengembalian dana yang perlu menggunakannya.

3. InstallmentPayment.java

```
4. package shop.payment;
5.
6. public interface InstallmentPayment extends PaymentMethod {
7.
8.     int MAX_MONTHS = 12;
9.
10.    default long monthlyInstallment(long amount, int months) {
11.        if (months <= 0 || months > MAX_MONTHS) {
12.            throw new IllegalArgumentException("Invalid
   installment months");
13.        }
14.
15.        return (amount + months - 1) / months;
16.    }
17.
18.    default int maxInstallmentMonths() {
19.        return MAX_MONTHS;
20.    }
21.
22.    default String policy() {
23.        return "Cicilan maksimal " + maxInstallmentMonths() +
   " bulan";
24.    }
```

```
}
```

Interface `InstallmentPayment` digunakan untuk metode pembayaran yang menyediakan fasilitas cicilan. Nilai `MAX_MONTHS` menentukan batas maksimal cicilan, yaitu 12 bulan. Method `monthlyInstallment()` digunakan untuk menghitung jumlah pembayaran setiap bulan berdasarkan total tagihan dan jumlah bulan cicilan. Sebelum menghitung, program memeriksa apakah jumlah bulan yang dimasukkan sesuai dengan batas yang ditentukan. Interface ini juga memiliki method `maxInstallmentMonths()` dan `policy()` untuk memberikan informasi mengenai ketentuan cicilan.

4. CreditCardPayment.java

```
package shop.payment;

import shop.StudentInfo;

public class CreditCardPayment implements RefundablePayment, InstallmentPayment {

    private final String cardHolder;
    private long limit;
    private long used;

    public CreditCardPayment(String cardHolder, long limit) {
        if (cardHolder == null || cardHolder.isBlank()) {
            throw new IllegalArgumentException("Card holder must not be blank");
        }
        if (limit < 0) {
            throw new IllegalArgumentException("Limit must not be negative");
        }

        this.cardHolder = cardHolder;
        this.limit = limit;
    }

    @Override
    public String getName() {
        return "Kartu Kredit (" + cardHolder + ")";
    }

    @Override
    public boolean pay(long amount) {
```

```
34.         if (amount <= 0 || amount > limit - used) {
35.             return false;
36.         }
37.
38.         used += amount;
39.         return true;
40.     }
41.
42.     @Override
43.     public long adminFee(long amount) {
44.         return amount * StudentInfo.CARD_ADMIN_PERCENT / 100;
45.     }
46.
47.     @Override
48.     public boolean refund(long amount) {
49.         if (amount <= 0 || amount > used) {
50.             return false;
51.         }
52.
53.         used -= amount;
54.         return true;
55.     }
56.
57.     @Override
58.     public String policy() {
59.         return RefundablePayment.super.policy() + "; "
60.                 + InstallmentPayment.super.policy();
61.     }
62. }
```

Class `CreditCardPayment` digunakan untuk melakukan pembayaran menggunakan kartu kredit. Class ini mengimplementasikan dua interface sekaligus, yaitu `RefundablePayment` dan `InstallmentPayment`, sehingga kartu kredit memiliki fitur refund dan cicilan. Data yang disimpan meliputi nama pemegang kartu, batas kredit, dan jumlah kredit yang sudah digunakan. Method `adminFee()` menghitung biaya administrasi berdasarkan persentase yang terdapat pada `StudentInfo`. Pada method `policy()`, program menggabungkan kebijakan refund dan cicilan. Hal ini dilakukan untuk mengatasi konflik karena kedua interface memiliki method default dengan nama yang sama.

5. BankTransferPayment.java

```
6. package shop.payment;
7.
8. import shop.StudentInfo;
```

```
9.
10. public class CreditCardPayment implements RefundablePayment,
    InstallmentPayment {
11.
12.     private final String cardHolder;
13.     private long limit;
14.     private long used;
15.
16.     public CreditCardPayment(String cardHolder, long limit) {
17.         if (cardHolder == null || cardHolder.isBlank()) {
18.             throw new IllegalArgumentException("Card holder
    must not be blank");
19.         }
20.         if (limit < 0) {
21.             throw new IllegalArgumentException("Limit must not
    be negative");
22.         }
23.
24.         this.cardHolder = cardHolder;
25.         this.limit = limit;
26.     }
27.
28.     @Override
29.     public String getName() {
30.         return "Kartu Kredit (" + cardHolder + ")";
31.     }
32.
33.     @Override
34.     public boolean pay(long amount) {
35.         if (amount <= 0 || amount > limit - used) {
36.             return false;
37.         }
38.
39.         used += amount;
40.         return true;
41.     }
42.
43.     @Override
44.     public long adminFee(long amount) {
45.         return amount * StudentInfo.CARD_ADMIN_PERCENT / 100;
46.     }
47.
48.     @Override
49.     public boolean refund(long amount) {
50.         if (amount <= 0 || amount > used) {
51.             return false;
52.         }
```

```
53.
54.         used -= amount;
55.         return true;
56.     }
57.
58.     @Override
59.     public String policy() {
60.         return RefundablePayment.super.policy() + "; "
61.                 + InstallmentPayment.super.policy();
62.     }
63. }
```

Class `BankTransferPayment` digunakan untuk menangani pembayaran melalui transfer bank. Class ini mengimplementasikan interface `PaymentMethod` dan menyimpan nama bank yang digunakan. Pada bagian constructor, terdapat pemeriksaan agar nama bank tidak kosong. Method `getName()` menampilkan nama metode pembayaran beserta nama bank. Method `pay()` memastikan jumlah pembayaran harus lebih dari nol, sedangkan `adminFee()` mengambil biaya administrasi tetap dari class `StudentInfo`, yaitu sebesar Rp3.000.

6. EWalletPayment.java

```
7. package shop.payment;
8.
9. public class EWalletPayment implements RefundablePayment {
10.
11.     private final String name;
12.     private long balance;
13.
14.     public EWalletPayment(String name, long balance) {
15.         if (name == null || name.isBlank()) {
16.             throw new IllegalArgumentException("Name must not
    be blank");
17.         }
18.         if (balance < 0) {
19.             throw new IllegalArgumentException("Balance must
    not be negative");
20.         }
21.
22.         this.name = name;
23.         this.balance = balance;
24.     }
25.
26.     @Override
27.     public String getName() {
```

```
28.         return name;
29.     }
30.
31.     public long getBalance() {
32.         return balance;
33.     }
34.
35.     @Override
36.     public boolean pay(long amount) {
37.         if (amount <= 0 || amount > balance) {
38.             return false;
39.         }
40.
41.         balance -= amount;
42.         return true;
43.     }
44.
45.     @Override
46.     public boolean refund(long amount) {
47.         if (amount <= 0) {
48.             return false;
49.         }
50.
51.         balance += amount;
52.         return true;
53.     }
54. }
```

Class `EWalletPayment` digunakan untuk melakukan pembayaran menggunakan saldo dompet digital. Class ini mengimplementasikan interface `RefundablePayment`, sehingga dapat melakukan pembayaran dan pengembalian dana. Data yang disimpan terdiri dari nama pengguna dan jumlah saldo. Method `pay()` akan mengurangi saldo jika jumlah pembayaran lebih dari nol dan saldo mencukupi. Sementara itu, method `refund()` akan menambahkan saldo apabila jumlah pengembalian lebih dari nol. Class ini tidak perlu membuat ulang method `policy()` karena sudah memperoleh implementasi default dari `RefundablePayment`.

7. CheckoutService.java

```
8. package shop.payment;
9.
10. public class EWalletPayment implements RefundablePayment {
11.
12.     private final String name;
```

```
13.     private long balance;
14.
15.     public EWalletPayment(String name, long balance) {
16.         if (name == null || name.isBlank()) {
17.             throw new IllegalArgumentException("Name must not
    be blank");
18.         }
19.         if (balance < 0) {
20.             throw new IllegalArgumentException("Balance must
    not be negative");
21.         }
22.
23.         this.name = name;
24.         this.balance = balance;
25.     }
26.
27.     @Override
28.     public String getName() {
29.         return name;
30.     }
31.
32.     public long getBalance() {
33.         return balance;
34.     }
35.
36.     @Override
37.     public boolean pay(long amount) {
38.         if (amount <= 0 || amount > balance) {
39.             return false;
40.         }
41.
42.         balance -= amount;
43.         return true;
44.     }
45.
46.     @Override
47.     public boolean refund(long amount) {
48.         if (amount <= 0) {
49.             return false;
50.         }
51.
52.         balance += amount;
53.         return true;
54.     }
55. }
```

Class `CheckoutService` bertugas mengatur proses transaksi pembelian produk. Method `subtotal()` menghitung seluruh harga produk, sedangkan `totalWeight()` menghitung berat produk yang dapat dikirim. Method `total()` menjumlahkan harga produk dengan biaya pengiriman. Selanjutnya, method `grandTotal()` menghitung total akhir dengan menambahkan biaya administrasi sesuai metode pembayaran yang digunakan. Method `checkout()` menjalankan proses pembayaran setelah memeriksa apakah daftar produk tersedia. Class ini juga memiliki method `refund()` untuk memproses pengembalian dana melalui metode pembayaran yang mendukung fitur refund.

8. StudentInfo.java

```
9. package shop;
10.
11.public final class StudentInfo {
12.    public static final String NIM = "2511531006";
13.    public static final String NAME = "Lexi Mulia Yunaspi";
14.
15.    public static final int CARD_ADMIN_PERCENT = 1;
16.    public static final long TRANSFER_ADMIN_FEE = 3000;
17.    public static final int SMS_MAX_LENGTH = 106;
18.
19.    private StudentInfo() {
20.        // constants only; no objects
21.    }
22.}
23.
```

Class `StudentInfo` digunakan untuk menyimpan informasi mahasiswa dan beberapa nilai tetap yang dibutuhkan dalam program. Class ini menyimpan NIM dan nama mahasiswa sebagai identitas. Selain itu, terdapat konstanta untuk persentase biaya administrasi kartu kredit, biaya tetap transfer bank, dan batas maksimal panjang pesan SMS. Kata kunci `final` pada class mencegah class ini diwariskan oleh class lain. Constructor dibuat `private` agar class tidak dapat dibuat menjadi objek karena hanya berfungsi sebagai tempat penyimpanan nilai tetap

2.7 T2 – Sistem Notifikasi

Tugas ini membahas pembuatan sistem notifikasi untuk memberikan informasi kepada pelanggan melalui beberapa saluran, yaitu email, SMS, dan WhatsApp. Program menggunakan interface dan abstract class agar setiap saluran

notifikasi memiliki aturan yang sama, tetapi tetap dapat menjalankan proses pengiriman sesuai jenisnya.

1. **CustomerContact.java**

```
2. package shop.notification;
3. 
4. public final class CustomerContact {
5. 
6.     private final String name;
7.     private final String email;
8.     private final String phone;
9. 
10.     public CustomerContact(String name, String email, String
   phone) {
11.         if (name == null || name.isBlank()) {
12.             throw new IllegalArgumentException("Name must not
   be blank");
13.         }
14. 
15.         this.name = name;
16.         this.email = email == null ? "" : email;
17.         this.phone = phone == null ? "" : phone;
18.     }
19. 
20.     public String getName() {
21.         return name;
22.     }
23. 
24.     public String getEmail() {
25.         return email;
26.     }
27. 
28.     public String getPhone() {
29.         return phone;
30.     }
31. }
32. 
```

Class `CustomerContact` digunakan untuk menyimpan informasi kontak pelanggan yang akan menerima notifikasi. Data yang disimpan terdiri dari nama, alamat email, dan nomor telepon. Pada constructor, program memeriksa agar nama pelanggan tidak kosong. Jika email atau nomor telepon tidak diisi, nilainya akan diubah menjadi teks kosong agar tidak menyebabkan nilai `null`. Atribut dibuat

`final` sehingga data kontak tidak dapat diubah setelah objek dibuat. Method getter digunakan untuk mengambil informasi kontak pelanggan.

2. NotificationChannel.java

```
3. package shop.notification;
4.
5. public interface NotificationChannel {
6.
7.     String channelName();
8.
9.     boolean send(CustomerContact contact, String message);
10.}
```

Interface `NotificationChannel` digunakan sebagai aturan dasar bagi semua saluran notifikasi. Interface ini memiliki dua method, yaitu `channelName()` dan `send()`. Method `channelName()` digunakan untuk mengetahui nama saluran yang digunakan, sedangkan method `send()` digunakan untuk mengirim pesan kepada pelanggan. Setiap class notifikasi yang menggunakan interface ini harus menyediakan implementasi kedua method tersebut. Dengan adanya interface ini, sistem dapat menggunakan berbagai jenis saluran notifikasi dengan aturan yang sama.

3. AbstractChannel.java

```
4. package shop.notification;
5.
6. public abstract class AbstractChannel implements
   NotificationChannel {
7.
8.     private int sentCount = 0;
9.     private String lastDelivered = "";
10.
11.    @Override
12.    public final boolean send(CustomerContact contact, String
   message) {
13.
14.        if (contact == null || message == null ||
   message.isBlank()) {
15.            return false;
16.        }
17.
18.        String recipient = resolveRecipient(contact);
19.
20.        if (recipient == null || recipient.isBlank()) {
```

```
            return false;
        }

        String preparedMessage = prepare(message);
        boolean success = deliver(recipient, preparedMessage);

        if (success) {
            sentCount++;
            lastDelivered = preparedMessage;
        }

        return success;
    }

    protected abstract String resolveRecipient(CustomerContact
contact);

    protected abstract boolean deliver(String recipient,
String message);

    protected String prepare(String message) {
        return message;
    }

    public int getSentCount() {
        return sentCount;
    }

    public String getLastDelivered() {
        return lastDelivered;
    }
}

```

Abstract class `AbstractChannel` merupakan class dasar untuk semua saluran notifikasi, seperti email, SMS, dan WhatsApp. Method `send()` digunakan untuk menjalankan proses pengiriman pesan dan dibuat `final` agar tidak dapat diubah oleh class turunannya. Sebelum pesan dikirim, program memeriksa data kontak dan isi pesan serta memastikan alamat atau nomor tujuan tersedia. Method `resolveRecipient()` dan `deliver()` harus dibuat oleh setiap class turunan sesuai jenis notifikasinya. Jika pengiriman berhasil, program akan menambah jumlah pesan terkirim dan menyimpan pesan terakhir melalui variabel `lastDelivered`.

4. **EmailChannel.java**

```
package shop.notification;

public class EmailChannel extends AbstractChannel {

    public EmailChannel() {
    }

    @Override
    public String channelName() {
        return "EMAIL";
    }

    @Override
    protected String resolveRecipient(CustomerContact contact)
{
        String email = contact.getEmail();

        if (email != null && email.contains("@")) {
            return email;
        }

        return null;
    }

    @Override
    protected boolean deliver(String recipient, String
message) {
        System.out.println("[EMAIL to " + recipient + "] " +
message);
        return true;
    }
}
```

Class `EmailChannel` digunakan untuk mengirimkan notifikasi kepada pelanggan melalui email. Class ini merupakan turunan dari `AbstractChannel`, sehingga mengikuti proses pengiriman pesan yang sudah disediakan oleh class tersebut. Method `channelName()` digunakan untuk memberikan nama saluran notifikasi, yaitu EMAIL. Method `resolveRecipient()` memeriksa alamat email pelanggan dan memastikan alamat tersebut memiliki tanda `@`. Selanjutnya, method `deliver()` menampilkan pesan beserta alamat email tujuan ke layar sebagai simulasi pengiriman.

5. **SmsChannel.java**

```
6. package shop.notification;
7.
8. public class SmsChannel extends AbstractChannel {
9.
10.     private final int maxLength;
11.
12.     public SmsChannel(int maxLength) {
13.         this.maxLength = maxLength;
14.     }
15.
16.     @Override
17.     public String channelName() {
18.         return "SMS";
19.     }
20.
21.     @Override
22.     protected String resolveRecipient(CustomerContact contact)
    {
23.         String phone = contact.getPhone();
24.
25.         if (phone != null && phone.startsWith("08")) {
26.             return phone;
27.         }
28.
29.         return null;
30.     }
31.
32.     @Override
33.     protected String prepare(String message) {
34.         if (message.length() > maxLength) {
35.             return message.substring(0, maxLength - 3) +
    "...";
36.         }
37.
38.         return message;
39.     }
40.
41.     @Override
42.     protected boolean deliver(String recipient, String
    message) {
43.         System.out.println("[SMS to " + recipient + "] " +
    message);
44.         return true;
45.     }
46. }
47.
```

Class `SmsChannel` digunakan untuk mengirimkan notifikasi kepada pelanggan melalui SMS. Class ini merupakan turunan dari `AbstractChannel` dan memiliki atribut `maxLength` untuk menentukan batas panjang pesan. Method `channelName()` memberikan nama saluran, yaitu SMS. Method `resolveRecipient()` memeriksa nomor telepon pelanggan agar diawali dengan angka 08. Method `prepare()` akan memotong pesan jika panjangnya melebihi batas yang ditentukan dan menambahkan tanda titik tiga di bagian akhir. Selanjutnya, method `deliver()` menampilkan pesan beserta nomor telepon tujuan sebagai simulasi pengiriman SMS.

6. WhatsAppChannel.java

```
7. package shop.notification;
8. 
9. public class WhatsAppChannel extends AbstractChannel {
10. 
11.     public WhatsAppChannel() {
12.     }
13. 
14.     @Override
15.     public String channelName() {
16.         return "WA";
17.     }
18. 
19.     @Override
20.     protected String resolveRecipient(CustomerContact contact)
    {
21.         String phone = contact.getPhone();
22. 
23.         if (phone == null) {
24.             return null;
25.         }
26. 
27.         boolean validLocal =
28.                 phone.startsWith("08") && phone.length() >=
    10;
29. 
30.         boolean validInternational =
31.                 phone.startsWith("+62") && phone.length() >=
    10;
32. 
33.         if (validLocal || validInternational) {
34.             return phone;
35.         }
```

```
36.
37.         return null;
38.     }
```

Class `WhatsAppChannel` digunakan untuk mengirimkan notifikasi kepada pelanggan melalui WhatsApp. Class ini merupakan turunan dari `AbstractChannel` sehingga menggunakan proses pengiriman yang sudah disediakan oleh class induknya. Method `channelName()` memberikan nama saluran, yaitu WA. Method `resolveRecipient()` memeriksa nomor telepon agar sesuai dengan format nomor lokal yang diawali 08 atau format internasional yang diawali +62. Method `prepare()` menambahkan tulisan `[Toko UMKM]` di awal pesan. Sementara itu, method `deliver()` menampilkan pesan beserta nomor WhatsApp tujuan sebagai simulasi pengiriman.

7. NotificationService.java

```
8. package shop.notification;
9.
10.import java.util.ArrayList;
11.import java.util.List;
12.
13.public class NotificationService {
14.
15.     private final List<NotificationChannel> channels =
16.             new ArrayList<>();
17.
18.     public void register(NotificationChannel channel) {
19.         if (channel != null) {
20.             channels.add(channel);
21.         }
22.     }
23.
24.     public int broadcast(CustomerContact contact, String
    message) {
25.         int successCount = 0;
26.
27.         for (NotificationChannel channel : channels) {
28.             if (channel.send(contact, message)) {
29.                 successCount++;
30.             }
31.         }
32.
33.         return successCount;
34.     }
35.}
```

Class `NotificationService` digunakan untuk mengelola beberapa saluran notifikasi dalam satu sistem. Class ini menyimpan daftar saluran notifikasi menggunakan `ArrayList`. Method `register()` digunakan untuk mendaftarkan saluran notifikasi baru selama objek yang dimasukkan tidak bernilai `null`. Method `broadcast()` digunakan untuk mengirim pesan kepada pelanggan melalui seluruh saluran yang sudah didaftarkan. Setiap pengiriman yang berhasil akan dihitung dan ditambahkan ke variabel `successCount`. Dengan demikian, sistem dapat mengirim notifikasi melalui beberapa saluran tanpa harus memanggil setiap class secara terpisah.

Kesimpulan T2: Sistem notifikasi dibuat menggunakan interface dan abstract class agar proses pengiriman dapat digunakan oleh beberapa saluran sekaligus. Pada kode yang diberikan, pengiriman masih berupa simulasi melalui tampilan console, belum mengirim email, SMS, atau WhatsApp secara nyata.

2.8 T3 – Code Review dan Refactoring

Tugas ini bertujuan untuk menemukan kekurangan pada desain program `BadShop.java`, kemudian memperbaikinya melalui proses refactoring. Perbaikan dilakukan agar program memiliki pembagian tanggung jawab yang lebih jelas, mudah dikembangkan, dan lebih sesuai dengan konsep pemrograman berorientasi objek.

1. **BadShop.java**

```
2. package review;
3. 
4. import java.util.ArrayList;
5. import java.util.List;
6. 
7. /*
8.  * T3 - CODE REVIEW
9.  * This file compiles and runs, but its design has several
   problems.
10. * Find them, explain them, and refactor (see the module for
   instructions).
11. */
12. 
13.interface StoreOperations {
14.    boolean pay(long amount);
```

```
15.
16.     boolean refund(long amount);
17.
18.     long ship(int weightGrams);
19.
20.     String printInvoice();
21. }
22.
23. class GiftVoucher implements StoreOperations {
24.     private double balance = 100000.0;
25.
26.     @Override
27.     public boolean pay(long amount) {
28.         if (amount > balance) {
29.             return false;
30.         }
31.         balance -= amount;
32.         return true;
33.     }
34.
35.     @Override
36.     public boolean refund(long amount) {
37.         throw new UnsupportedOperationException("Voucher
    cannot be refunded");
38.     }
39.
40.     @Override
41.     public long ship(int weightGrams) {
42.         throw new UnsupportedOperationException("Voucher is
    not shipped");
43.     }
44.
45.     @Override
46.     public String printInvoice() {
47.         return "Voucher, remaining balance " + balance;
48.     }
49. }
50.
51. abstract class Courier {
52.     public long cost(String type, int kilograms) {
53.         switch (type) {
54.             case "REGULAR":
55.                 return kilograms * 10_000L;
56.             case "EXPRESS":
57.                 return kilograms * 18_000L;
58.             case "SAME_DAY":
59.                 return kilograms * 30_000L;
```

```
60.             default:
61.                 return 0;
62.         }
63.     }
64.}
65.
66.class JneCourier extends Courier {
67.}
68.
69.class Cart {
70.     private final List<Item> items = new ArrayList<>();
71.
72.     class Item {
73.         private final String name;
74.         private final long price;
75.         private final int quantity;
76.
77.         Item(String name, long price, int quantity) {
78.             this.name = name;
79.             this.price = price;
80.             this.quantity = quantity;
81.         }
82.
83.         long subtotal() {
84.             return price * quantity;
85.         }
86.
87.         String getName() {
88.             return name;
89.         }
90.     }
91.
92.     void add(Item item) {
93.         items.add(item);
94.     }
95.
96.     long total() {
97.         long total = 0;
98.         for (Item item : items) {
99.             total += item.subtotal();
100.              }
101.              return total;
102.          }
103.      }
104.
105.      class Address {
106.          private String street;
```

```
107.         private String city;
108.
109.         Address(String street, String city) {
110.             this.street = street;
111.             this.city = city;
112.         }
113.
114.         public void setStreet(String street) {
115.             this.street = street;
116.         }
117.
118.         public void setCity(String city) {
119.             this.city = city;
120.         }
121.
122.         @Override
123.         public String toString() {
124.             return street + ", " + city;
125.         }
126.     }
127.
128.     class SimpleOrder {
129.         private final String id;
130.         private final Address address;
131.
132.         SimpleOrder(String id, Address address) {
133.             this.id = id;
134.             this.address = address;
135.         }
136.
137.         Address getAddress() {
138.             return address;
139.         }
140.
141.         @Override
142.         public String toString() {
143.             return id + " -> " + address;
144.         }
145.     }
146.
147.     interface Discount {
148.         default long apply(long price) {
149.             if (this instanceof MemberDiscount) {
150.                 return price - price * 5 / 100;
151.             } else if (this instanceof FlashSaleDiscount) {
152.                 return price - price * 30 / 100;
153.             }
```

```
154.             return price;
155.         }
156.     }
157.
158.     class MemberDiscount implements Discount {
159.     }
160.
161.     class FlashSaleDiscount implements Discount {
162.     }
163.
164.     public class BadShop {
165.         public static void main(String[] args) {
166.             Cart cart = new Cart();
167.             cart.add(cart.new Item("Kopi Gayo", 85_000, 2));
168.             System.out.println("Total cart: " +
    cart.total());
169.
170.             Courier courier = new JneCourier();
171.             System.out.println("Ongkir EXPRES 2 kg: " +
    courier.cost("EXPRES", 2));
172.
173.             Address home = new Address("Jl. Sudirman 10",
    "Padang");
174.             SimpleOrder first = new SimpleOrder("ORD-1",
    home);
175.             SimpleOrder second = new SimpleOrder("ORD-2",
    home);
176.             second.getAddress().setCity("Bukittinggi");
177.             System.out.println(first);
178.             System.out.println(second);
179.
180.             Discount flashSale = new FlashSaleDiscount();
181.             System.out.println("Harga flash sale: " +
    flashSale.apply(100_000));
182.
183.             StoreOperations voucher = new GiftVoucher();
184.             System.out.println(voucher.pay(30_000) + " | " +
    voucher.printInvoice());
185.             System.out.println("Ongkir voucher: " +
    voucher.ship(100));
186.         }
187.     }
```

Class `BadShop` merupakan program awal yang digunakan untuk mempelajari kesalahan dalam perancangan program berorientasi objek. Di dalamnya terdapat interface `StoreOperations` yang memiliki terlalu banyak

tanggung jawab, yaitu pembayaran, refund, pengiriman, dan pencetakan invoice. Class `GiftVoucher` juga dipaksa memiliki fitur refund dan pengiriman meskipun fitur tersebut tidak sesuai dengan kegunaannya. Perhitungan biaya kurir masih menggunakan `switch`, sedangkan sistem diskon menggunakan `instanceof`. Selain itu, class `Address` masih dapat diubah sehingga perubahan alamat dapat memengaruhi pesanan lain yang menggunakan objek yang sama. Program ini menjadi bahan untuk latihan memperbaiki desain menggunakan konsep pemrograman berorientasi objek.

2. BadShopRefactored.java

```
3. package review.fixed;
4. 
5. import java.util.ArrayList;
6. import java.util.List;
7. 
8. // Interface khusus pembayaran
9. interface Payable {
10.     boolean pay(long amount);
11.}
12. 
13.// Interface khusus refund
14.interface Refundable {
15.     boolean refund(long amount);
16.}
17. 
18.// Interface khusus pengiriman
19.interface Shippable {
20.     long ship(int weightGrams);
21.}
22. 
23.// Voucher hanya membutuhkan kemampuan pembayaran
24.class GiftVoucher implements Payable {
25. 
26.     private double balance = 100000.0;
27. 
28.     @Override
29.     public boolean pay(long amount) {
30.         if (amount <= 0 || amount > balance) {
31.             return false;
32.         }
33. 
34.         balance -= amount;
35.         return true;
```

```
36.     }
37.
38.     public String printInvoice() {
39.         return "sisa Rp" + (long) balance;
40.     }
41.}
42.
43.// Abstract class dengan perhitungan khusus setiap kurir
44.abstract class Courier {
45.
46.     public abstract long calculateCost(int kilograms);
47.}
48.
49.class JneCourier extends Courier {
50.
51.     @Override
52.     public long calculateCost(int kilograms) {
53.         return kilograms * 10000L;
54.     }
55.}
56.
57.class ExpressCourier extends Courier {
58.
59.     @Override
60.     public long calculateCost(int kilograms) {
61.         return kilograms * 18000L;
62.     }
63.}
64.
65.class SameDayCourier extends Courier {
66.
67.     @Override
68.     public long calculateCost(int kilograms) {
69.         return kilograms * 30000L;
70.     }
71.}
72.
73.// Cart dengan static nested class Item
74.class Cart {
75.
76.     private final List<Item> items = new ArrayList<>();
77.
78.     static class Item {
79.
80.         private final String name;
81.         private final long price;
82.         private final int quantity;
```

```
83.
84.         Item(String name, long price, int quantity) {
85.             this.name = name;
86.             this.price = price;
87.             this.quantity = quantity;
88.         }
89.
90.         long subtotal() {
91.             return price * quantity;
92.         }
93.     }
94.
95.     void add(Item item) {
96.         items.add(item);
97.     }
98.
99.     long total() {
100.             long total = 0;
101.
102.             for (Item item : items) {
103.                 total += item.subtotal();
104.             }
105.
106.             return total;
107.         }
108.     }
109.
110.     // Address dibuat immutable
111.     final class Address {
112.
113.         private final String street;
114.         private final String city;
115.
116.         Address(String street, String city) {
117.             this.street = street;
118.             this.city = city;
119.         }
120.
121.         @Override
122.         public String toString() {
123.             return street + ", " + city;
124.         }
125.     }
126.
127.     class SimpleOrder {
128.
129.         private final String id;
```

```
130.         private final Address address;
131.
132.         SimpleOrder(String id, Address address) {
133.             this.id = id;
134.             this.address = address;
135.         }
136.
137.         @Override
138.         public String toString() {
139.             return id + " -> " + address;
140.         }
141.     }
142.
143.     // Interface diskon menggunakan polymorphism
144.     interface Discount {
145.
146.         long apply(long price);
147.     }
148.
149.     class MemberDiscount implements Discount {
150.
151.         @Override
152.         public long apply(long price) {
153.             return price - price * 5 / 100;
154.         }
155.     }
156.
157.     class FlashSaleDiscount implements Discount {
158.
159.         @Override
160.         public long apply(long price) {
161.             return price - price * 30 / 100;
162.         }
163.     }
164.
165.     // Class utama
166.     public class BadShopRefactored {
167.
168.         public static void main(String[] args) {
169.
170.             // 1. Menghitung total cart
171.             Cart cart = new Cart();
172.
173.             cart.add(new Cart.Item("Kopi Gayo", 85000, 2));
174.
175.             System.out.println("Total cart: " +
    cart.total());
```

```
176.
177.             // 2. Menghitung ongkir Express
178.             Courier courier = new ExpressCourier();
179.
180.             System.out.println("Ongkir Ekspres 2 kg: "
181.                     + courier.calculateCost(2));
182.
183.             // 3. Menguji alamat pesanan
184.             Address home1 = new Address(
185.                     "Jl. Sudirman 10", "Padang");
186.
187.             Address home2 = new Address(
188.                     "Jl. Sudirman 10", "Bukittinggi");
189.
190.             SimpleOrder first = new SimpleOrder("ORD-1",
    home1);
191.             SimpleOrder second = new SimpleOrder("ORD-2",
    home2);
192.
193.             System.out.println(first);
194.             System.out.println(second);
195.
196.             // 4. Menguji diskon
197.             Discount flashSale = new FlashSaleDiscount();
198.
199.             System.out.println("Harga flash sale: "
200.                     + flashSale.apply(100000));
201.
202.             // 5. Menguji pembayaran voucher
203.             Payable voucher = new GiftVoucher();
204.
205.             boolean success = voucher.pay(30000);
206.
207.             GiftVoucher giftVoucher = (GiftVoucher) voucher;
208.
209.             System.out.println(success + " | "
210.                     + giftVoucher.printInvoice());
211.         }
212.     }
213.
```

Class `BadShopRefactored` merupakan hasil perbaikan dari program `BadShop` sebelumnya. Perbaikan dilakukan dengan memisahkan interface berdasarkan tugasnya, yaitu `Payable`, `Refundable`, dan `Shippable`. Perhitungan

biaya pengiriman diperbaiki menggunakan abstract class `Courier` dan method `calculateCost()` yang diterapkan oleh setiap jenis kurir. Class `Cart.Item` diubah menjadi static nested class karena tidak membutuhkan data dari objek `Cart`. Selain itu, class `Address` dibuat immutable agar data alamat tidak dapat diubah setelah objek dibuat. Penggunaan polymorphism pada sistem diskon juga membuat program lebih mudah dikembangkan.

2.9 T4 – Refleksi

Tugas ini membahas pemahaman mengenai penggunaan abstract class, interface, dan nested class dalam pemrograman Java. Pemilihan jenis class harus disesuaikan dengan kebutuhan program agar kode lebih terstruktur dan mudah dikembangkan.

1. Perbedaan Abstract Class dan Interface

No.	Kondisi	Pilihan	Contoh dalam program
1	Membutuhkan atribut dan constructor yang digunakan bersama oleh beberapa class.	Abstract class	Product` sebagai class dasar untuk `PhysicalProduct` dan `DigitalProduct`.
2	Membutuhkan beberapa kemampuan yang dapat digunakan oleh satu class sekaligus.	Interface	CreditCardPayment` menggunakan `RefundablePayment` dan `InstallmentPayment`.
3	Memiliki rumus dasar yang tidak boleh diubah oleh class turunannya.	Abstract class dengan method `final	ShippingService.calculateCost()`.
4	Membutuhkan aturan yang dapat dikembangkan dengan menambahkan method baru.	Interface	PaymentMethod`.

Abstract class digunakan ketika beberapa class memiliki atribut, constructor, atau proses dasar yang sama. Sementara itu, interface digunakan untuk memberikan aturan atau kemampuan yang dapat diterapkan oleh berbagai class. Satu class dapat menggunakan lebih dari satu interface sekaligus, seperti `CreditCardPayment` yang memiliki kemampuan refund dan cicilan. Pemilihan

keduanya harus disesuaikan dengan kebutuhan program agar struktur kode lebih mudah dipahami dan dikembangkan.

2. Jenis Nested Class

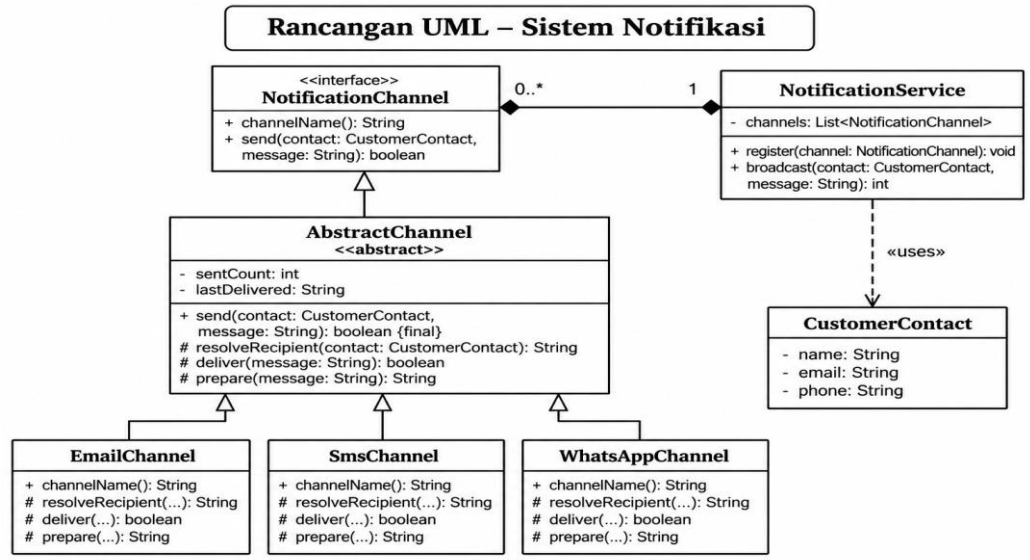
No.	Kondisi	Jenis class	Contoh dalam program
1	Class tambahan tidak membutuhkan akses ke objek luar.	Static nested class	Cart.Item`.
2	Class tambahan membutuhkan akses ke data milik objek luar.	Inner class	Order.Receipt`.
3	Implementasi hanya digunakan sekali dan tidak membutuhkan nama class tersendiri.	Anonymous class	Implementasi `PaymentMethod` pada `ShopApp`.

Nested class adalah class yang dibuat di dalam class lain. Static nested class digunakan jika class tambahan tidak membutuhkan data dari objek luar, sedangkan inner class digunakan jika membutuhkan akses terhadap data objek yang membungkusnya. Anonymous class digunakan untuk membuat implementasi class atau interface secara langsung tanpa harus membuat file class tersendiri. Dengan memahami ketiga jenis tersebut, programmer dapat menentukan struktur class yang lebih sesuai dengan kebutuhan program.

Jawaban pertanyaan T1

1. EWalletPayment tidak perlu meng-override policy() karena class tersebut hanya mengimplementasikan RefundablePayment. Dengan demikian, hanya ada satu method default policy() yang diwarisi sehingga tidak terjadi konflik.
2. Java tidak memilih salah satu method default secara otomatis karena kedua interface memberikan implementasi untuk method dengan tanda tangan yang sama. Pemilihan otomatis dapat menimbulkan ketidakjelasan aturan yang harus digunakan. Oleh sebab itu, CreditCardPayment harus meng-override suatu policy() dan memanggil RefundablePayment.super.policy() serta InstallmentPayment.super.policy().

Rancangan UML sistem notifikasi



Relasi UML: EmailChannel, SmsChannel, dan WhatsAppChannel mewarisi AbstractChannel. AbstractChannel mengimplementasikan NotificationChannel. NotificationService menyimpan kumpulan objek NotificationChannel, sedangkan proses pengiriman menggunakan data CustomerContact.

Jawaban pertanyaan T2

1. Interface dan abstract class digunakan bersama karena keduanya memiliki fungsi berbeda. NotificationChannel menjadi kontrak umum yang dapat dipakai berbagai jenis kanal, sedangkan AbstractChannel menyediakan alur pengiriman yang sama dan menyimpan data pencatatan. Dengan demikian, kanal baru cukup mengisi bagian yang berbeda.
2. Jika send() tidak dibuat final, class turunan dapat mengganti seluruh alur pengiriman dan melewati pemeriksaan kontak, validasi pesan, persiapan pesan, atau pencatatan. Hal ini dapat membuat perilaku antar-kanal tidak konsisten.
3. NotificationService menyimpan List<NotificationChannel> agar bergantung pada interface, bukan pada satu abstract class. Desain ini lebih fleksibel karena semua objek yang memenuhi kontrak NotificationChannel dapat didaftarkan, termasuk kanal log berbentuk anonymous class.

Tabel hasil code review T3

```
achments\praktikum6\abstraction-2511531006\bin' 'review.BadShop'
Total cart: 170000
Ongkir EXPRES 2 kg: 0
ORD-1 -> Jl. Sudirman 10, Bukittinggi
ORD-2 -> Jl. Sudirman 10, Bukittinggi
Harga flash sale: 70000
true | Voucher, remaining balance 70000.0
Exception in thread "main" java.lang.UnsupportedOperationException: Voucher is not shipped
        at review.GiftVoucher.ship(BadShop.java:41)
        at review.BadShop.main(BadShop.java:184)
```

Program in menghasilkan total belanja Rp170.000 dan diskon flash sale sebesar Rp30.000 sehingga harga menjadi Rp70.000. Pembayaran voucher berhasil dengan sisa saldo Rp70.000.Namun,ongkos kirim menghasilkan Rp0 dan program mengalami error karena voucher dipaksa menjalankan fungsi pengiriman.Hal ini menunjukkan masih terdapat kesalahan dalam desain program yang perlu diperbaiki melalui refactoring.

No.	Lokasi	Masalah	Dampak nyata	Perbaikan yang diusulkan
1	StoreOperations / GiftVoucher	Satu interface memaksa class memiliki pay, refund, ship, dan printInvoice.	GiftVoucher tidak mendukung refund dan pengiriman sehingga melempar UnsupportedOperationException.	Pisahkan kontrak menjadi Payable, Refundable, dan Shippable; voucher cukup implements Payable.
2	Courier.cost()	Tarif kurir ditentukan dengan switch berdasarkan String.	Salah ketik seperti EXPRES menghasilkan ongkir 0 dan class kurir tidak benar-benar memiliki perilaku khusus.	Gunakan abstract method calculateCost() dan implementasi pada tiap subclass kurir.
3	Cart.Item	Item dibuat sebagai inner class non-static padahal tidak memakai data Cart.	Setiap Item membawa referensi tersembunyi ke objek Cart yang tidak diperlukan.	Ubah menjadi static nested class Cart.Item.
4	Address / SimpleOrder	Address dapat diubah dan objek alamat yang sama dipakai dua pesanan.	Mengubah alamat pesanan kedua ikut mengubah alamat pesanan pertama.	Jadikan Address immutable dengan field final dan tanpa setter, atau salin objek alamat.
5	Discount.apply()	Interface default memeriksa jenis objek	Penambahan jenis diskon baru mengharuskan perubahan pada method yang sama dan	Jadikan apply() sebagai kontrak dan implementasikan

		menggunakan instanceof.	mengurangi manfaat polymorphism.	rumus pada tiap class diskon.
6	GiftVoucher.pay()	Tidak memeriksa amount <= 0 dan memakai double untuk saldo uang.	Pembayaran bernilai nol/negatif dapat mengubah saldo secara tidak benar dan double tidak tepat untuk nominal rupiah.	Validasi amount > 0 dan gunakan long untuk satuan rupiah.
7	Courier.cost()	Class JneCourier tidak mengubah perilaku class induk.	Nama subclass memberi kesan ada layanan khusus, tetapi tarif tetap ditentukan oleh parameter String.	Berikan override calculateCost() pada JneCourier, ExpressCourier, dan SameDayCourier.

Refactoring pada BadShopRefactored.java memperbaiki beberapa masalah utama, yaitu memisahkan interface berdasarkan kemampuan, mengganti switch tarif kurir dengan polymorphism, menjadikan Cart.Item sebagai static nested class, membuat Address immutable, dan menerapkan polymorphism pada diskon. Berdasarkan contoh keluaran modul, ongkir Ekspres untuk 2 kg menjadi Rp36.000, alamat ORD-1 tetap di Padang, alamat ORD-2 di Bukittinggi, dan program tidak lagi berhenti karena voucher dipaksa melakukan pengiriman.

2.10 Pengujian Program dan Analisis Hasil

Pengujian pada laporan ini mengikuti format yang diminta modul (bagian E.5) dan dibagi menjadi dua kelompok. Kelompok pertama adalah pengujian error compiler untuk L1, L3, L4, T1.a, dan T1.e. Baris kode uji pada kelompok ini sengaja dibuat salah untuk membuktikan aturan abstraction di Java, jadi pesan error yang muncul justru hasil yang diharapkan. Kelompok kedua adalah pemeriksaan otomatis memakai TugasCheck, SelfCheck, dan keluaran TugasApp. Setiap gambar diberi nomor sesuai urutan pembahasan.

2.10.1 Pengujian L1 – Abstract Class

Pada L1, dilakukan pengujian dengan mencoba membuat objek secara langsung dari abstract class Product. Baris kode uji yang digunakan adalah:

```
Product p = new Product("X", "X", 1000);
```

Hasil kompilasi menunjukkan bahwa Product tidak dapat diinstansiasi karena masih memiliki abstract method category(). Objek Product harus dibuat melalui subclass konkret, seperti PhysicalProduct atau DigitalProduct.

```java
package shop.app;

import shop.model.Product;

public class L1Error {

    Run | Debug
    public static void main(String[] args) {
        Product p = new Product("X", "X", 1000);
    }
}
```

Gambar 2.10.1 Kode pengujian instansiasi abstract class Product.

```
PS C:\Users\Hype AMD\OneDrive\Attachments\praktikum6\abstraction-2511531006>  & 'C:\Users\Hype AMD\.vscode\extensions\redhat.java-1.5
8.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Hype AMD\OneDrive\Att
achments\praktikum6\abstraction-2511531006\bin' 'shop.app.L1Error'
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
        Cannot instantiate the type Product

        at shop.app.L1Error.main(L1Error.java:7)
PS C:\Users\Hype AMD\OneDrive\Attachments\praktikum6\abstraction-2511531006> javac -cp bin -d bin src/shop/app/L1Error.java
src\shop\app\L1Error.java:7: error: Product is abstract; cannot be instantiated
        Product p = new Product("X", "X", 1000);
                    ^
1 error
```

Gambar 2.10.2 Pesan error compiler pada pengujian L1.

2.10.2 Pengujian L3 – Interface Inheritance dan Refund

Pada L3, dilakukan pengujian dengan memanggil method refund() menggunakan objek BankTransferPayment. Pemanggilan yang diuji adalah:

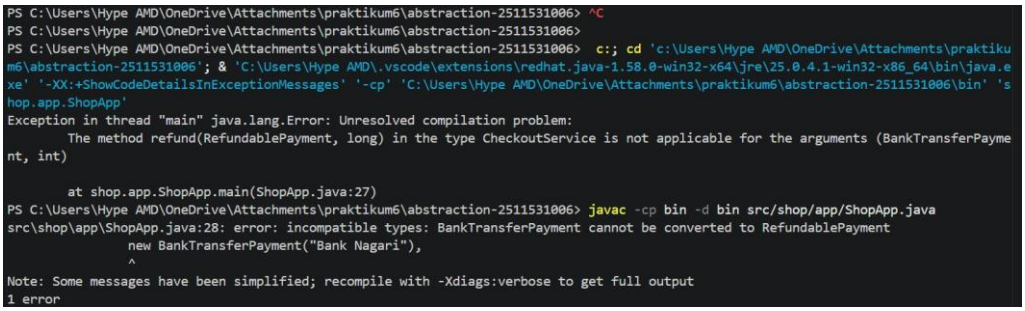
```
checkout.refund(new BankTransferPayment("Bank Nagari"), 1_000);
```

Compiler menolak pemanggilan tersebut karena parameter method refund() pada CheckoutService harus bertipe RefundablePayment, sedangkan BankTransferPayment hanya mengimplementasikan PaymentMethod. Dengan demikian, transfer bank tidak dapat digunakan untuk proses refund.



```
26        // BUKTI ERROR L3
27        checkout.refund(new BankTransferPayment(bankName: "Bank Nagari"),amount: 1_000);
28
29        Address address = new Address(street: "Jl. Moh Hatta No. 10", city: "Padang", postalCode: "25121");
30        Order order = new Order(orderId: "ORD-0001", address);
31        order.addLine(coffee, quantity: 2);
32        order.addLine(ebook, quantity: 1);
33
34        Order.Receipt receipt = order.new Receipt();
35        System.out.println(receipt.toText());
36
37        PaymentMethod voucher = new PaymentMethod() {
38            private long remaining = 50_000;
39
40            @Override
41            public String getName() {
42                return "Voucher Promo Hari Ini";
43            }
44
45            @Override
46            public boolean pay(long amount) {
47                if (amount <= 0 || amount > remaining) {
48                    return false;
49                }
50
51                remaining -= amount;
52                return true;
53            }
54        };
55        System.out.println("\n" + voucher.getName()+ " untuk e-book berhasil? "+ checkout.checkout(List.of(ebook), r
56    }
```

Gambar 2.10.3 Kode pemanggilan refund dengan BankTransferPayment.



```
PS C:\Users\Hype AMD\OneDrive\Attachments\praktikum6\abstraction-2511531006> ^C
PS C:\Users\Hype AMD\OneDrive\Attachments\praktikum6\abstraction-2511531006>
PS C:\Users\Hype AMD\OneDrive\Attachments\praktikum6\abstraction-2511531006>  c:; cd 'c:\Users\Hype AMD\OneDrive\Attachments\praktiku
m6\abstraction-2511531006'; & 'C:\Users\Hype AMD\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.e
xe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Hype AMD\OneDrive\Attachments\praktikum6\abstraction-2511531006\bin' 's
hop.app.ShopApp'
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
        The method refund(RefundablePayment, long) in the type CheckoutService is not applicable for the arguments (BankTransferPayme
nt, int)

        at shop.app.ShopApp.main(ShopApp.java:27)
PS C:\Users\Hype AMD\OneDrive\Attachments\praktikum6\abstraction-2511531006> javac -cp bin -d bin src/shop/app/ShopApp.java
src\shop\app\ShopApp.java:28: error: incompatible types: BankTransferPayment cannot be converted to RefundablePayment
                new BankTransferPayment("Bank Nagari"),
                ^
Note: Some messages have been simplified; recompile with -Xdiags:verbose to get full output
1 error
```

Gambar 2.10.4 Pesan error compiler pada pengujian L3.

2.10.3 Pengujian L4 – Final Method

Pada L4, dilakukan pengujian dengan membuat subclass FreeShipping yang mencoba melakukan override terhadap method calculateCost(int) milik ShippingService. Kode yang diuji adalah:

```
public long calculateCost(int weightGrams) {
    return 0;
}
```

Hasil kompilasi menunjukkan bahwa calculateCost(int) tidak dapat di-override karena method tersebut diberi modifier final. Subclass tetap dapat menentukan tarif melalui ratePerKg(), tetapi tidak boleh mengubah algoritma perhitungan ongkos kirim pada class induk.

```java
package shop.shipping;

public class FreeShipping extends ShippingService {

    public FreeShipping() {
        super(name: "Gratis");
    }

    @Override
    protected long ratePerKg() {
        return 0;
    }

    @Override
    public long calculateCost(int weightGrams) {
        return 0;
    }
}
```

Gambar 2.10.5 Kode FreeShipping yang mencoba override calculateCost(int).

```
PS C:\Users\Hype AMD\OneDrive\Attachments\praktikum6\abstraction-2511531006> javac -cp bin -d bin src/shop/shipping/L4Error.java
src\shop\shipping\L4Error.java:15: error: calculateCost(int) in L4Error cannot override calculateCost(int) in ShippingService
    public long calculateCost(int weightGrams) {
                ^
  overridden method is final
1 error
```

Gambar 2.10.6 Pesan error compiler pada pengujian L4.

2.10.4 Pengujian T1.a – Abstract Method adminFee()

Pada T1.a, method long adminFee(long amount); ditambahkan ke interface PaymentMethod sebagai abstract method biasa (Gambar 2.10.7), lalu proyek dikompilasi. Hasilnya muncul 4 error pada 4 tempat, yaitu class EWalletPayment, BankTransferPayment, CreditCardPayment, dan anonymous class voucher di ShopApp (Gambar 2.10.8).

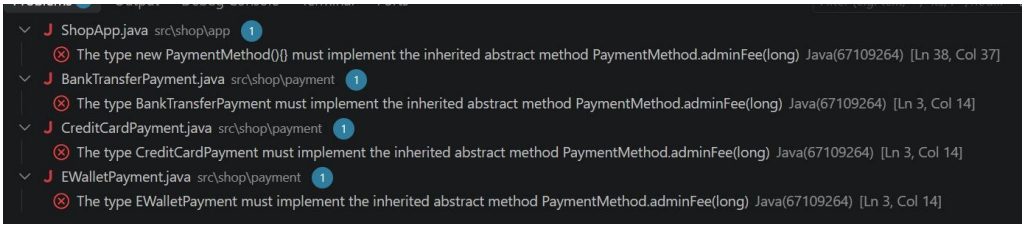
```java
package shop.payment;

public interface PaymentMethod {
    String getName();

    boolean pay(long amount);

    long adminFee(long amount);
}
```

Gambar 2.10.7 Deklarasi adminFee(long) sebagai abstract method pada PaymentMethod.

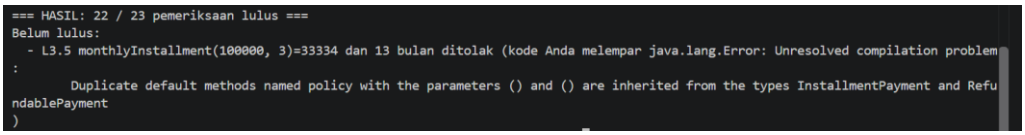


Gambar 2.10.8 Error compiler pada ShopApp, BankTransferPayment, CreditCardPayment, dan EWalletPayment.

Keempat error berbunyi sama, yaitu tipe tersebut harus mengimplementasikan abstract method adminFee(long) yang diwarisi dari PaymentMethod. Anonymous class voucher ikut rusak karena ia juga mengimplementasikan PaymentMethod secara langsung. Anonymous class tetap termasuk class konkret yang hanya tidak punya nama, sehingga wajib mengisi semua abstract method dari interface. Masalah ini selesai pada T1.b ketika adminFee() diubah menjadi default method.

2.10.5 Pengujian T1.e – Konflik Default Method policy()

Pada T1.e, default method policy() ditambahkan ke RefundablePayment dan InstallmentPayment, sedangkan CreditCardPayment yang mengimplementasikan keduanya belum meng-override policy(). Java menolak keadaan ini dengan pesan Duplicate default methods named policy, yang artinya CreditCardPayment mewarisi dua default method bertanda tangan sama dari dua interface berbeda (Gambar 2.10.9). Error ini juga membuat SelfCheck L3.5 gagal sehingga hasilnya 22/23.



Gambar 2.10.9 Error konflik default method policy() pada CreditCardPayment, terlihat di hasil SelfCheck L3.5.

Konflik diselesaikan dengan meng-override policy() di CreditCardPayment, lalu isinya menggabungkan kedua kebijakan lewat dua pemanggilan berikut:

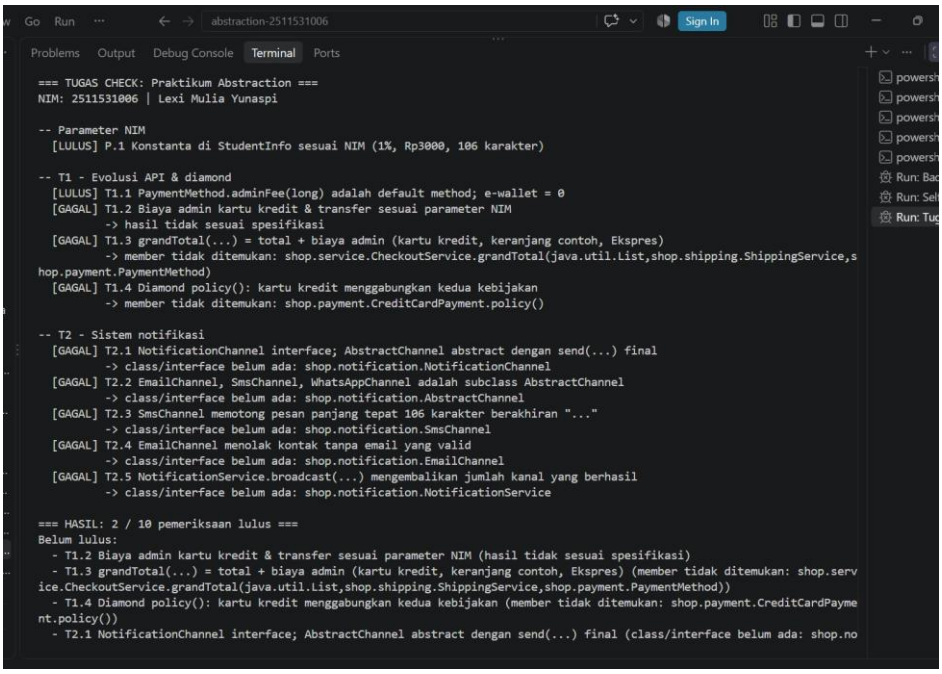
RefundablePayment.super.policy()

InstallmentPayment.super.policy()

Setelah itu error hilang dan SelfCheck kembali lulus penuh (Subbab 2.10.7).

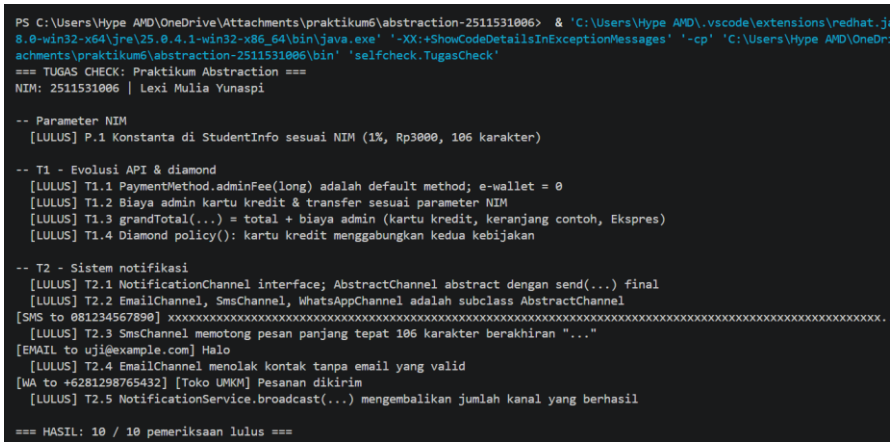
2.10.6 Hasil TugasCheck – T1 dan T2

TugasCheck dijalankan lewat selfcheck.TugasCheck untuk NIM 2511531006. Pemeriksaan P.1 lulus karena konstanta di StudentInfo sesuai NIM, yaitu CARD_ADMIN_PERCENT 1%,TRANSFER_ADMIN_FEE Rp3000,dan SMS_MAX_LENGTH 106.Pada pemeriksaan awal, hanya 2 dari 10 yang lulus (P.1 dan T1.1).T1.2 sampai T1.4 dan seluruh T2 masih gagal karena biaya admin, grandTotal(), policy(), serta package shop.notification belum dikerjakan (Gambar 2.10.10).



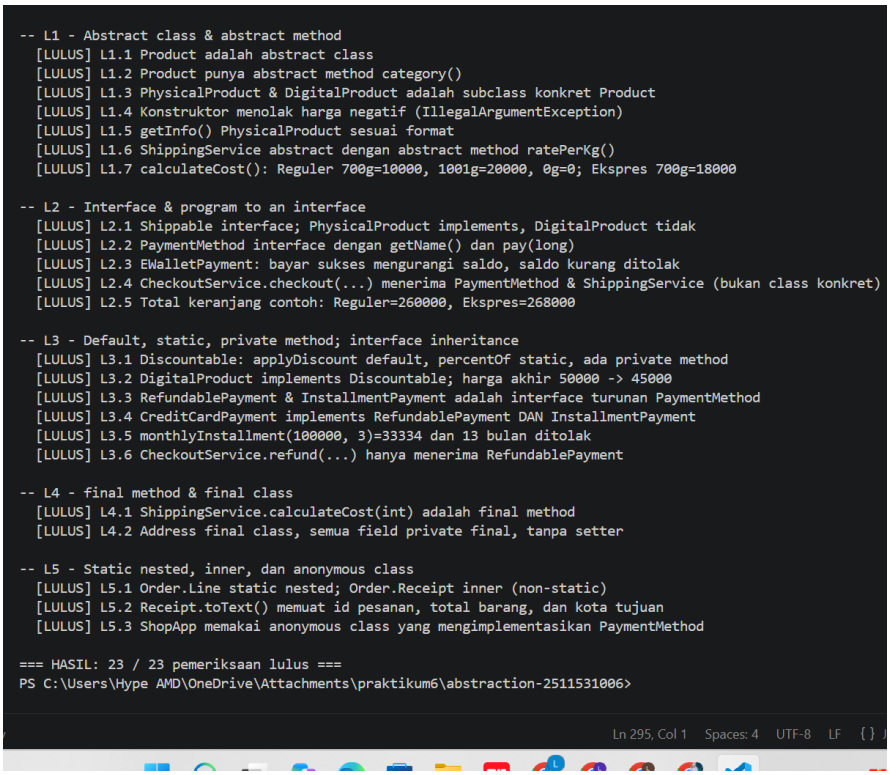
Gambar 2.10.10 Hasil TugasCheck awal: 2 dari 10 pemeriksaan lulus.

Setelah T1 dan T2 selesai dikerjakan, TugasCheck menunjukkan hasil akhir 10 dari 10 pemeriksaan lulus (Gambar 2.10.11).


PS C:\Users\Hype AMD\OneDrive\Attachments\praktikum6\abstraction-2511531006> & 'C:\Users\Hype AMD\.vscode\extensions\redhat.j
8.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Hype AMD\OneDr
achments\praktikum6\abstraction-2511531006\bin' 'selfcheck.TugasCheck'
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511531006 | Lexi Mulia Yunaspi
-- Parameter NIM
[LULUS] P.1 Konstanta di StudentInfo sesuai NIM (1%, Rp3000, 106 karakter)
-- T1 - Evolusi API & diamond
[LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
[LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
[LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
[LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan
-- T2 - Sistem notifikasi
[LULUS] T2.1 NotificationChannel interface; AbstractChannel abstract dengan send(...) final
[LULUS] T2.2 EmailChannel, SmsChannel, WhatsAppChannel adalah subclass AbstractChannel
[SMS to 081234567890] xxxx

2.10.7 Hasil SelfCheck – Latihan L1 sampai L5

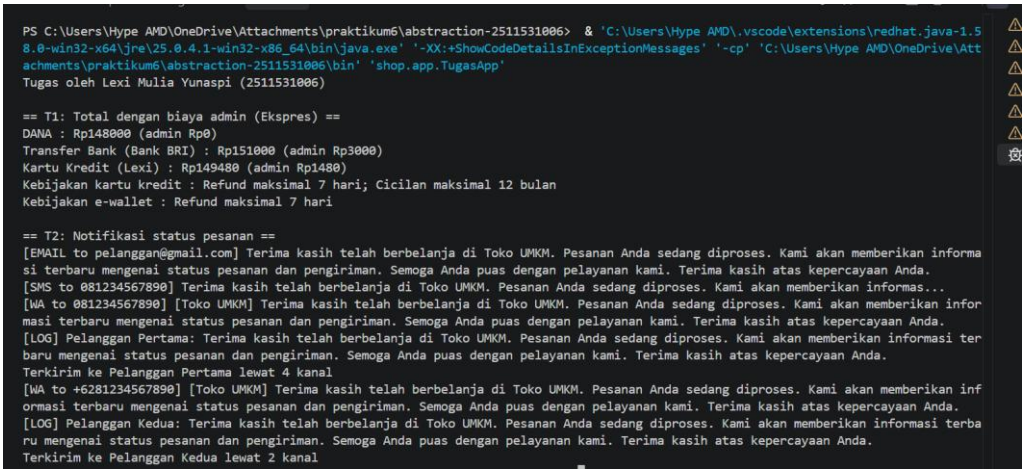
SelfCheck dijalankan untuk memeriksa seluruh latihan L1 sampai L5. Hasil akhirnya 23 dari 23 pemeriksaan lulus, sesuai target pada modul (Gambar 2.10.12).



Gambar 2.10.12 Hasil akhir SelfCheck: 23 dari 23 pemeriksaan lulus.

2.10.8 Keluaran TugasApp

TugasApp dijalankan untuk menampilkan grand total keranjang contoh dengan kurir Ekspres untuk tiga metode pembayaran, kebijakan policy() kartu kredit dan e-wallet, serta hasil pengiriman notifikasi ke dua kontak lewat kanal yang terdaftar (Gambar 2.10.13). Hasilnya, grand total Ekspres untuk e-wallet Rp148000 (admin Rp0), transfer bank Rp151000 (admin Rp3000), dan kartu kredit Rp149480 (admin Rp1480). Kontak pertama menerima pesan lewat 4 kanal, sedangkan kontak kedua hanya lewat 2 kanal karena tidak punya email dan nomornya berawalan +62.



Gambar 2.10.13 Keluaran program TugasApp.

2.10.9 Analisis Hasil Pengujian

Pengujian error compiler pada L1, L3, L4, T1.a, dan T1.e menunjukkan bahwa compiler Java menjaga aturan abstraction sebelum program dijalankan. Abstract class tidak bisa diinstansiasi, parameter bertipe interface membatasi objek yang boleh dikirim, method final tidak bisa di-override, abstract method baru memaksa semua class implementasi (termasuk anonymous class) ikut berubah, dan dua default method yang bentrok harus diselesaikan dengan override yang jelas.

Dari sisi pemeriksaan otomatis, TugasCheck memperoleh 10 dari 10 dan SelfCheck memperoleh 23 dari 23 pemeriksaan lulus. Hal ini menandakan bahwa semua spesifikasi latihan dan tugas rumah yang diperiksa program sudah terpenuhi. Nilai SelfCheck sempat turun menjadi 22/23 saat konflik policy() belum diselesaikan, lalu kembali penuh setelah method tersebut di-override.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan latihan L1 sampai L5 dan tugas rumah T1 sampai T4, dapat disimpulkan bahwa abstraction membantu menyusun program dengan memisahkan hal yang perlu diketahui pengguna dari detail pelaksanaannya. Abstract class cocok digunakan untuk menyediakan atribut dan perilaku dasar yang sama, sedangkan interface digunakan untuk menetapkan kemampuan atau kontrak yang dapat diterapkan oleh berbagai class.

Penerapan default method dapat membantu mengembangkan interface tanpa mengharuskan seluruh class lama menambahkan implementasi baru. Pewarisan interface memungkinkan satu class memiliki beberapa kemampuan, tetapi konflik default method harus diselesaikan dengan override secara jelas. Modifier final dapat menjaga aturan penting agar tidak diubah subclass. Sementara itu, nested class membantu menempatkan class pendukung sesuai hubungannya dengan class utama.

Melalui tugas code review, terlihat bahwa desain yang baik perlu menghindari interface yang terlalu luas, ketergantungan pada pemeriksaan tipe, data yang mudah berubah tanpa kendali, serta penggunaan inner class yang tidak diperlukan. Pemisahan tanggung jawab dan polymorphism membuat program lebih mudah dipahami dan dikembangkan.

3.2 Saran

Mahasiswa sebaiknya tidak hanya menyalin kode, tetapi memahami fungsi setiap class, interface, dan method serta mencoba menjalankan program secara bertahap. Pesan error compiler perlu dibaca sebagai petunjuk untuk menemukan sumber masalah. Sebelum laporan dikumpulkan, seluruh kode, parameter berdasarkan NIM, hasil SelfCheck, TugasCheck, dan bukti pengujian perlu diperiksa kembali agar sesuai dengan proyek yang benar-benar dijalankan.

DAFTAR PUSTAKA

Deitel, P., & Deitel, H. (2017). Java How to Program, Early Objects (Edisi ke-11). Pearson.

Program Studi S1 Informatika, Universitas Andalas. (2026). Modul Praktikum OOP 6 – Abstraction. Universitas Andalas.